EasyTrip - A Bus Ticket Booking System for Infivro

Rakesh Al Yadin

ID# 22203121

A Practicum in the Partial Fulfillment of the Requirements

for the Award of Bachelor of Computer Science and Engineering (BCSE)

Department of Computer Science and Engineering
College of Engineering and Technology
IUBAT—International University of Business Agriculture and Technology

Spring 2026

EasyTrip - A Bus Ticket Booking System for Infivro

Rakesh Al Yadin

A Practicum in the Partial Fulfillment of the Requirements for the Award of Bachelor of Computer Science and Engineering (BCSE)

The practicum has been examined and approved,

Prof. Dr. Utpal Kanti Das
Professor and Chairman, Dept of CSE
Dean, ISCSE

Shahinur Alam
Co-supervisor, Coordinator and Assistant Professor

Rafiqul Islam Munna
Supervisor and Lecturer

Department of Computer Science and Engineering
College of Engineering and Technology
IUBAT—International University of Business Agriculture and Technology

Spring 2026

Letter of Transmittal

12 May 2026

The Chair

Practicum Defense Committee

Department of Computer Science and Engineering

IUBAT—International University of Business Agriculture and Technology

4 Embankment Drive Road, Sector 10, Uttara Model Town

Dhaka 1230, Bangladesh.

Subject: Letter of Transmittal.

Dear Sir,

I am pleased to present to you my practicum report titled “EasyTrip – A Bus Ticket Booking System” as required by IUBAT for the partial fulfillment of the requirements for the award of Bachelor of Computer Science and Engineering. It was indeed a great opportunity for me to work on this project to actualize my theoretical knowledge into practice. I am eagerly anticipating your thoughtful evaluation of my report.

I would like to express my gratitude for the opportunity to further my education at your esteemed university.

Yours sincerely,

____

Rakesh Al Yadin

22203121

Organization’s Certificate

Place your organization’s certificate scanned copy here.

Student's Declaration

This is to certify that the work presented in this report titled, "EasyTrip – A Bus Ticket Booking System for Infivro", is done by me for the partial fulfillment of the requirements for the Award of Bachelor of Computer Science and Engineering (BCSE). It is the outcome of my work carried out under the supervision of Rafiqul Islam Munna, Lecturer, Department of Computer Science and Engineering, International University of Business Agriculture and Technology.

Rakesh Al Yadin
22203121

Supervisor's Certification

This is to certify that Practicum report on "EasyTrip – A Bus Ticket Booking System" has been carried out by Rakesh Al Yadin, bearing ID# 22203121, of IUBAT–International University of Business Agriculture and Technology, as a partial fulfillment of the requirement of practicum defense course. The report has been prepared under my guidance and is a record of the accomplished work, carried out successfully. To the best of my knowledge and as per her declaration, no parts of this report has been submitted anywhere for any degree, diploma or certification.

He is hereby permitted to submit the report. I am confident that his hard work, commitment, and dedication to this project will lead her to continued success, and I extend my best wishes for all her future professional and academic endeavors.

Rafiqul Islam Munna

Supervisor and Lecturer

Department of Computer Science and Engineering

IUBAT—International University of Business Agriculture and Technology

Abstract

The rapid growth of online transportation services and the increasing demand for convenient travel solutions have created significant challenges in traditional bus ticket management systems. Conventional ticket booking processes are often inefficient, time-consuming, and dependent on manual operations, causing seat management conflicts, booking errors, and inconvenience for passengers. To address these issues, this project presents a web-based Online Bus Ticket Management System named “EasyTrip - A Bus Ticket System” developed using React.js, NestJS, PostgreSQL, Prisma ORM, and SSLCommerz Payment Gateway integration.

The proposed system allows users to register, log in, search available buses, view schedules, select seats, book tickets online, and complete secure digital payments. A real-time seat synchronization mechanism ensures that seat availability is updated instantly among multiple users without requiring page refresh. The system also implements temporary seat locking and concurrency control mechanisms to prevent duplicate seat booking conflicts.

Additional features of the system include secure online payment processing, refund request and approval workflow, automated PDF ticket and invoice generation, QR-code-based ticket verification, password recovery functionality, and email notification services. SSLCommerz APIs were integrated for secure payment and refund processing, while PDF and QR code generation libraries were used for invoice and ticket verification purposes.

The system was developed following software engineering principles including requirement engineering, project planning, system analysis, software testing, and quality management. Multiple test cases were conducted to ensure system reliability, security, usability, and performance under concurrent user operations.

The proposed EasyTrip Bus Ticket System improves ticket booking efficiency, reduces manual management complexity, enhances passenger convenience, and provides a scalable foundation for future smart transportation management solutions.

Acknowledgments

First and foremost, I express my deepest gratitude to Almighty Allah for granting me the strength, patience, and determination to successfully complete this work.

I would like to extend my heartfelt appreciation to IUBAT – International University of Business Agriculture and Technology and pay my sincere respect to its founder, Late Professor Dr. Md. Alimullah Miyan, whose vision and dedication continue to inspire generations of students.

I am also grateful to the Department of Computer Science and Engineering (CSE) for providing the academic environment, facilities and guidance necessary to carry out this project. My special thanks go to the Dean and Chairman Prof. Dr. Utpal Kanti Das sir and our coordinators Shahinur Alam sir and Dr. Rashedul Islam Rana sir of the CSE Department for their continuous support and encouragement throughout our study period.

I express my profound gratitude to my supervisor, Rafiqul Islam Munna, for his invaluable guidance, patience and insightful suggestions. His constant encouragement and expert feedback played a crucial role in shaping this project and bringing it to completion.

Finally, I would like to express my heartfelt thanks to my family for their unconditional love, patience and moral support. Their constant encouragement and understanding were the foundation that kept me motivated throughout this journey.

Table of Contents

List of Figures

List of Tables

Chapter 1.
Introduction

The **Bus Ticket Management System** is a web-based software solution developed to modernize and simplify the process of bus ticket booking and transport operations. Traditional ticketing methods are often manual, time-consuming, and prone to errors in seat allocation, schedule handling, and record maintenance. This system addresses those limitations by providing a centralized platform where passengers can search routes, view schedules, check seat availability, and book tickets efficiently.

In addition to passenger services, the system supports administrative functions such as managing buses, routes, trip schedules, and booking records. By automating key operations, the project reduces paperwork, improves data accuracy, and enhances overall service quality. The Bus Ticket Management System is therefore designed as a practical and scalable approach to improving convenience for users and operational efficiency for transport providers.

1.1 Background of the Study

Transportation is one of the most essential services in everyday life, and bus travel remains a primary and affordable option for a large number of people. However, in many transport services, ticket booking and management are still handled through manual processes or partially digital systems. These traditional methods often create several problems, such as long waiting times, lack of accurate seat availability information, booking errors, poor record management, and limited communication between passengers and bus operators.

With the growth of digital technologies, users now expect faster, transparent, and more convenient services. An online bus ticket system can reduce these limitations by allowing passengers to check schedules, view available seats, and reserve tickets from anywhere. At the same time, it helps operators manage routes, buses, bookings, and customer data in a more organized and efficient way.

The Bus Ticket Booking System project was initiated to address these real-world challenges by developing a centralized web-based solution for ticket reservation and transport management. The study is therefore focused on analyzing existing ticketing issues and implementing a practical software system that improves operational efficiency, reduces manual workload, and enhances user experience for both passengers and administrators.

1.2 Methodology

The methodology for developing this Online Bus Ticket Booking System is based on the NestJS framework with Prisma ORM, providing a scalable and efficient architecture for building secure and maintainable web applications. This approach ensures smooth integration between backend services, database management, and API communication, resulting in a reliable, user-friendly, and responsive ticket booking platform. NestJS handles authentication, business logic, and RESTful API development, while Prisma ORM is used for efficient database operations with SQL Server. The system supports secure user authentication, trip management, seat booking, online payment processing, refund management, and ticket generation, ensuring a smooth and efficient experience for passengers and administrators.

1.2.1 Data Sources.

Data is central to the Bus Ticket Management System. The primary data source is a relational database (PostgreSQL) managed through Prisma ORM, which stores structured information such as user accounts, buses, routes, trips, schedules, seats, bookings, payments, refunds, and ticket-related records. The database structure ensures data integrity, consistency, and efficient query processing for booking and management operations.

Additional data is generated through user interactions within the system. Passengers provide information during registration, login, trip searching, seat selection, booking, payment, and refund requests. Administrators manage operational data including buses, routes, schedules, pricing, and booking activities. Payment transaction data is securely processed and stored through integrated payment services. The NestJS backend API handles validation, authentication, and business logic, while the React (Vite) frontend ensures real-time interaction and seamless communication between users and the system.

1.3 Objectives

The objectives of the proposed system have been divided in to two categories: Broad objectives and Specific objectives. These objectives are given below:

1.3.1 Broad Objective.

The broad objective of this project is to design and develop a web-based Bus Ticket Management System that digitizes and centralizes bus transportation operations. The system enables passengers to search for trips, reserve seats, and manage bookings efficiently, while allowing administrators to manage buses, routes, schedules, and booking-related activities in a structured manner. The project aims to reduce manual work, improve accuracy and transparency, and enhance the overall efficiency of transport service management.

1.3.2 Specific Objectives.

- To develop a secure user registration and authentication system with role-based access control.
- To allow passengers to search trips, view seat availability, and book tickets online.
- To implement booking, payment, and refund management functionalities efficiently.
- To provide an administrative system for managing buses, routes, trips, and schedules.
- To maintain reliable and structured data storage using a relational database system.
- To develop a responsive and user-friendly web application for efficient ticket management.

1.4 Process Model

The Agile Software Development Model was used to develop the Bus Ticket Management System because it provides an iterative and flexible approach to software development through continuous planning, development, testing, and improvement. In this project, the system was divided into several modules including authentication, trip management, seat booking, payment processing, refund management, and ticket generation. Each module was developed and tested incrementally to ensure better functionality, maintainability, and reliability of the system. Continuous testing and debugging helped identify and resolve issues efficiently, while the flexibility of the Agile model allowed easier modification and enhancement of features throughout the development process.

Reasons for Choosing Agile Model

- Supports incremental and modular development of the system.
- Allows continuous testing and debugging during development.
- Provides flexibility for adding or modifying features easily.
- Improves collaboration and version control through GitHub workflow.
- Helps identify and solve problems quickly during development.
- Suitable for web-based applications with multiple interconnected modules.

1.5 Feasibility Study

The feasibility study has been divided into three phases such as technical feasibility, economic feasibility, and operational feasibility. The proposed system is feasible in all these three phases as discussed below:

1.5.1 Technical Feasibility.

- The system uses modern and widely adopted technologies such as React, NestJS, Prisma ORM, and PostgreSQL.
- The selected technologies provide secure, reliable, and scalable web application development.
- Features such as authentication, booking management, payment processing, and refunds follow standard software engineering practices.
- The project can be developed using commonly available tools, IDEs, and version control systems like GitHub.
- No special hardware or proprietary software is required for development and deployment.

- The expected number of users and data volume are manageable within the project scope.
- Potential technical risks such as bugs, integration issues, and security concerns can be minimized through testing and debugging.

1.5.2 Economic Feasibility.

- The project is developed using free and open-source technologies such as React, NestJS, Prisma ORM, and PostgreSQL.
- No expensive proprietary software licenses or specialized hardware are required.
- Development and deployment costs are relatively low for academic or small-scale use.
- The system can run on affordable hosting services with low maintenance expenses.
- Digital ticket management helps reduce manual work and operational errors.
- The system improves efficiency and saves time for both passengers and administrators.
- The project can be scaled gradually based on future user demand and operational needs.

1.5.3 Operational Feasibility.

- The system supports real-world bus ticket booking and transportation management operations.
- Passengers can easily search trips, select seats, book tickets, and manage bookings online.
- Administrators can efficiently manage buses, routes, schedules, bookings, and payments from a centralized system.
- The web-based interface is simple, user-friendly, and requires minimal technical knowledge.

- Only basic training is needed for administrators and staff to operate the system effectively.
- Centralized booking and seat management reduce manual errors and booking conflicts.
- The system improves coordination, transparency, and overall operational efficiency in transport management.

1.6 Structure of the Report

The report is structured into ten chapters, each focusing on a specific aspect of the EasyTrip - A Bus Ticket Management System. Chapter 1 presents the introduction, background of the study, objectives, methodology, feasibility study, and overall project overview. Chapter 2 provides an overview of the organization, including its vision, mission, services, organizational structure, and the role performed during the practicum period. Chapter 3 discusses requirement engineering, requirement elicitation, requirement analysis, requirement specification, and the use case diagram of the system.

Chapter 4 focuses on system analysis and describes the applied software analysis patterns and activity diagrams for major system functionalities. Chapter 5 presents project management activities including risk identification, risk analysis, risk planning, and the RMMM plan. Chapter 6 explains project planning, function-oriented metrics, complexity analysis, function point calculation, effort estimation, and project scheduling.

Chapter 7 describes the system design including user interface design, data flow diagrams, and database design with the entity relationship diagram. Chapter 8 discusses system quality management, software testing processes, and test case results. Chapter 9 presents ethical considerations and sustainability issues related to the software development process. Finally, Chapter 10 concludes the report by summarizing the project outcomes, benefits, limitations, practicum value, and future enhancement plans for the EasyTrip - A Bus Ticket Management System.

Chapter 2.
Organizational Overview

Infivro is a fast-growing software development company in Bangladesh focused on delivering software products that are valuable, meaningful, and user-centric. The company has a skilled and experienced team dedicated to providing high-quality software and IT solutions to clients. Infivro approaches every project with seriousness and prioritizes delivering the best possible service. The organization has strong expertise in modern software technologies and undertakes projects in areas such as web development, mobile application development, network security, and e-commerce solutions. Infivro is well recognized for developing, designing, and re-engineering web-based business applications. Its primary goal is to create software that is efficient, affordable, and easy to use for both developers and end users.

2.1 Organization Vision

Infivro aims to create innovative and impactful software solutions that deliver real value to businesses and users. The company values a professional team that is motivated, skilled, and committed to growth, leadership, and excellence. The vision of Infivro includes:

- To establish the organization as a trusted and reputable IT company globally.
- To become a leading software development company worldwide.
- To be the top choice for individuals and organizations seeking software solutions.
- To deliver high-quality, innovative, and reliable software products that meet international standards.

2.2 Organization Mission

The mission of Infivro is to empower clients by providing efficient, cost-effective, and high-quality software solutions. The company emphasizes automation, optimization, and innovation to enhance performance and customer satisfaction. Core values include:

- Integrity
- Innovation
- Efficiency

- Customer Satisfaction
- Cost-effective and scalable solutions
- Faster project delivery through effective teamwork
- High-quality and reliable final products

Infivro is committed to continuous improvement, maintaining strong corporate ethics, supporting economic growth in Bangladesh, and building long-term trust with clients by exceeding expectation.

2.3 Organization Services

Infivro offers a comprehensive range of software and IT services to support digital transformation and business growth. Their services include Web Development, where responsive, scalable, and high-performance websites are built. They also provide Mobile Application Development for both Android and iOS platforms. Additionally, Softdeft delivers Custom Software Solutions tailored to meet specific business requirements. The company offers Cloud-Based Solutions to improve scalability, security, and system performance. Infivro also works on Enterprise Solutions, E-Commerce Platforms, and IT Consultancy Services, helping organizations enhance operational efficiency and achieve long-term success..

2.4 Organizational Structure

The organogram of the organization has shown in figure 2.1.

Figure 2.1 Organizational Structure

2.5 My Position in this Organization

As a Software Engineer Intern at Infivro, I am responsible for designing, developing, and maintaining modern and responsive websites and web applications. My duties include creating user-friendly interfaces, optimizing performance, and ensuring cross-device and cross-browser compatibility. I collaborate closely with designers, project managers, and other developers to deliver projects on time and according to requirements. Additionally, I continually update my knowledge of emerging technologies and industry trends to improve the quality, efficiency, and user experience of the software solutions provided by Infivro.

2.6 Address of the Organization

House/Flat # 155/6, Dhaka Cantonment, Manikdi, ECB chatar - Dhaka Cantonment

Chapter 3.
Requirement Engineering

Requirements engineering is a fundamental phase of the software development lifecycle that focuses on identifying, analyzing, and documenting the needs, expectations, and constraints of all stakeholders involved in the system. This process provides a structured approach for designing, developing, and evaluating software solutions to ensure they effectively address user needs. For the Bus Ticket Management System, requirements engineering ensures that the platform satisfies the functional and non-functional expectations of passengers and administrators while supporting scalability, security, usability, reliability, and maintainability. The requirements engineering process for this project can be broadly divided into three main phases:

Requirements Gathering: In this phase, information is collected from stakeholders including passengers, administrators, and transport operators through discussions, observation of existing ticket management systems, and analysis of similar online booking platforms. The goal is to identify essential features such as user authentication, trip searching, seat selection, online ticket booking, payment processing, refund handling, ticket generation, and administrative management.

Requirements Analysis and Prioritization: Collected requirements are analyzed, categorized, and prioritized based on importance, feasibility, and alignment with system objectives. Both functional and non-functional requirements are identified, while conflicts and redundancies are minimized. Feasibility analysis is also performed to ensure that the selected technologies such as React, NestJS, Prisma ORM, and PostgreSQL can effectively support system performance, security, scalability, and reliability.

Requirements Documentation: This phase involves preparing detailed documentation of the identified system requirements and related design specifications. UML diagrams, use case diagrams, activity diagrams, and database designs are created to represent system interactions and functionalities including trip management, booking operations, payment processing, refund management, and administrative activities.

Through this structured requirements engineering process, the Bus Ticket Management System is designed to provide a reliable, secure, scalable, and user-friendly solution that improves transportation management and online ticket booking operations.

3.1 Requirement Elicitation

Requirement elicitation is the process of identifying, gathering, and understanding the requirements of users and stakeholders for developing the Bus Ticket Management System. This phase focused on analyzing the problems of the traditional ticket booking process and determining the necessary functionalities required for an efficient online transportation management system. The elicitation process helped identify the expectations of passengers and administrators to ensure that the system meets operational and user requirements effectively.

Key activities performed during this phase include:

- Collecting Requirements
- Deploying Quality Function Techniques (QFD)

3.1.1 Collecting Requirements

Requirements were collected by analyzing existing online ticket booking platforms, observing transportation management processes, and identifying common problems faced in manual ticket booking systems. The collected information helped determine important system functionalities such as user authentication, trip searching, seat selection, online booking, payment processing, refund management, and ticket generation.

Feedback from potential users and administrators was also considered to ensure that the system provides a secure, reliable, and user-friendly experience for both passengers and transport authorities.

3.1.2 Deploying Quality Function Techniques (QFD)

Quality Function Deployment (QFD) techniques were used to translate user expectations into technical and functional system requirements. Important quality factors such as usability, security, reliability, performance, and scalability were identified and mapped into system functionalities.This process ensured that the Bus Ticket Management System provides efficient seat management, secure payment handling, accurate booking information, responsive user interaction, and smooth administrative operations while maintaining overall system quality and user satisfaction.

3.2 Standard Requirements

Standard requirements define the objectives and expected functionalities of the Bus Ticket Management System, as identified through stakeholder discussions and operational needs. Proper implementation of these requirements ensures user satisfaction, reliable booking operations, and efficient transport management. The key requirements of the system are described below:

User Management: Passengers and administrators must be able to register, log in, and manage their profiles securely through role-based access control.

Trip and Route Management: Administrators should be able to create, update, and manage buses, routes, stops, and trip schedules to ensure accurate transportation operations.

Trip Browsing and Search: Passengers should be able to search available trips based on source, destination, travel date, and schedule information.

Seat Availability and Selection: The system must display real-time seat availability and allow passengers to select seats before confirming bookings.

Booking Management: Passengers must be able to book tickets smoothly, receive booking confirmations, and access booking history information.

Ticket Cancellation: Passengers should be able to cancel bookings according to system policies, and the system should update seat status accordingly.

Authentication and Authorization: The platform should provide secure authentication and authorization mechanisms to protect user data and system operations.

Payment Integration: The system must support secure digital payment processing and maintain payment status records linked to bookings.

Refund Workflow: The system should support refund requests and processing workflows with clear status tracking such as pending, approved, rejected, or failed.

Administrative Monitoring: Administrators should be able to monitor bookings, payments, cancellations, refunds, and operational activities from a centralized management panel.

Notification and Confirmation: The system should provide clear confirmations and status notifications for booking, payment, cancellation, and refund-related activities.

Data Integrity and Reliability: The platform must maintain accurate and consistent records for users, trips, seats, bookings, payments, and refunds through a structured relational database system.

3.3 Requirement Analysis of the Bus Ticket Management System

Requirement analysis is a critical step in which all possible approaches to solving the identified transportation and ticket management problems are examined. The proposed solution must fulfill the needs of passengers and administrators while remaining flexible enough to support future improvements and additional functionalities. This section presents a detailed requirement analysis of the Bus Ticket Management System, focusing on technical, operational, and economic feasibility to ensure the system is practical, reliable, and efficient for real-world transportation management operations.

3.4 Requirement Specification of the Bus Ticket Management System

This section outlines the detailed requirements for the Bus Ticket Management System, including user requirements, system requirements, functional requirements, and non-functional requirements. A use-case diagram has been developed to illustrate the core interactions between users and the system.

3.4.1 User Requirements

- Passengers can register, log in, search trips, select seats, and book tickets online.
- Passengers can view booking history, payment status, and ticket details from their accounts.
- Passengers can cancel bookings and request refunds according to system policies.
- Administrators can manage buses, routes, trip schedules, bookings, payments, and refunds.

- Administrators can monitor booking activities and maintain operational records through an admin dashboard.
- All users can update and manage their personal profile information securely.
- The system provides secure authentication and role-based access control for passengers and administrators.
- The system supports secure online payment processing and payment status tracking.
- The platform provides real-time seat availability and booking confirmation notifications.
- The system maintains accurate booking, payment, and refund records using a relational database.
- The platform should be responsive and accessible from desktop and mobile devices for a consistent user experience.

3.4.2 Functional requirements

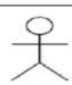
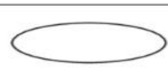
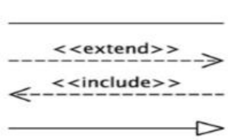
- The system shall allow users to register, log in, and log out securely.
- The system shall support role-based access control for passengers and administrators.
- The system shall allow administrators to add, update, and manage buses and related operational details.
- The system shall allow administrators to create, update, and manage routes, stops, and trip schedules.
- The system shall allow passengers to search available trips based on source, destination, and travel date.
- The system shall display trip information including departure time, fare, bus type/class, and available seats.
- The system shall allow passengers to select seats and complete ticket bookings online.

- The system shall generate and maintain booking records with statuses such as pending, confirmed, and cancelled.
- The system shall allow passengers to view booking history and ticket details from their accounts.
- The system shall allow passengers to cancel bookings according to system policies and constraints.
- The system shall process and maintain payment information associated with bookings.
- The system shall support refund requests and refund status tracking where applicable.
- The system shall allow administrators to monitor bookings, payments, cancellations, and refund activities.
- The system shall provide notifications and confirmation messages for booking, payment, cancellation, and refund-related events.
- The system shall maintain audit fields such as timestamps and status updates for important transactional records.

3.5 Use Case Diagram of the System

3.5.1 Use Case Symbols

Table 1: Use Case Symbol Representation

Symbol	Reference Name
	Actor
	Use Case
	Relationship

3.5.2 Use Case Diagram.

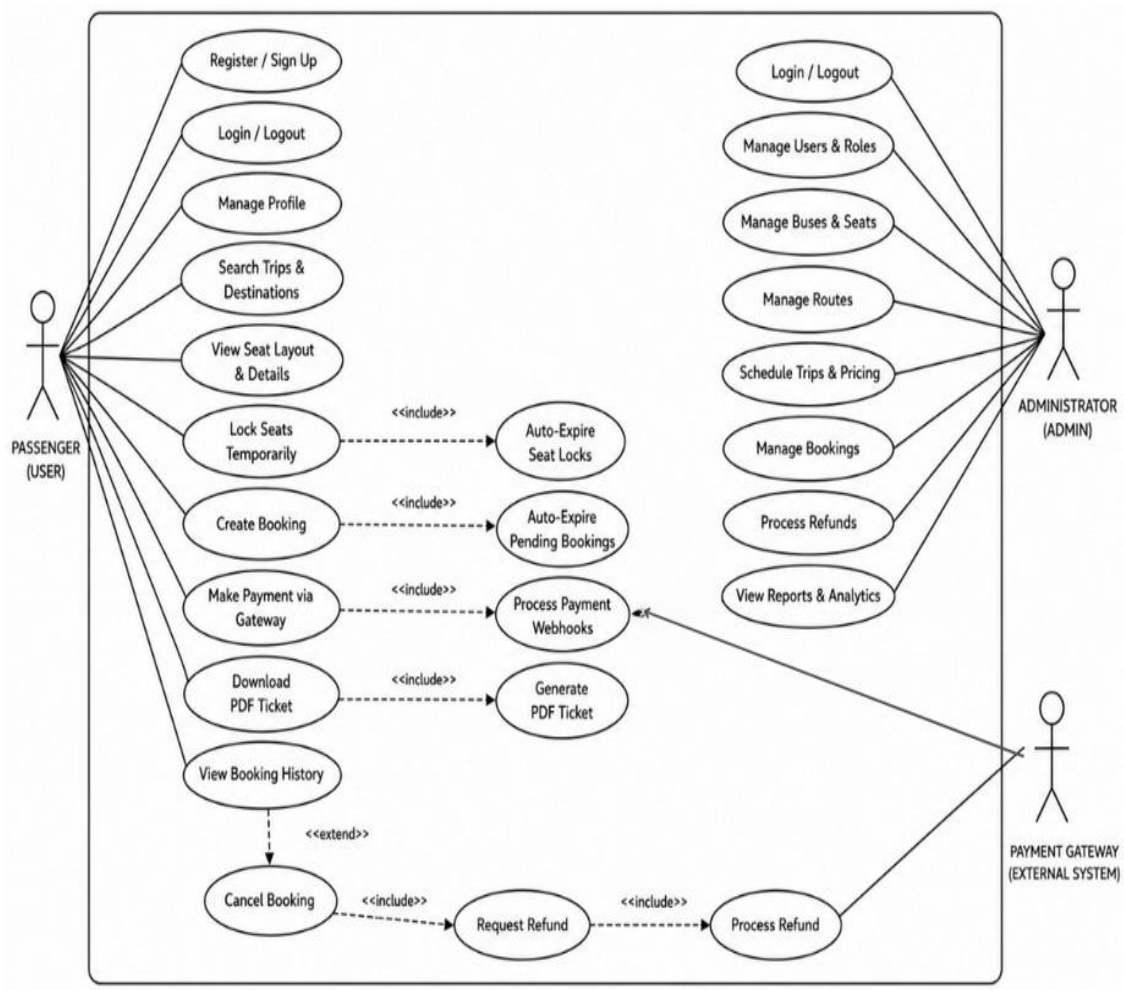


Figure 3.1 Use Case Diagram of the System.

Chapter 4.

Analysis

Software analysis patterns are reusable solutions to common software design challenges that help improve system organization, maintainability, and scalability. By applying these patterns, complex systems can be designed with clear separation of responsibilities, simplifying development, testing, and future enhancements. For example, the MVC (Model–View–Controller) pattern separates application logic, user interface, and data handling, making the system easier to manage and extend.

For the Bus Ticket Booking System developed at Infivro, the following software analysis patterns have been applied:

- Layered Architecture Pattern:
  - Presentation Layer (React Frontend): Responsible for user-facing interfaces such as trip searching, seat selection, booking pages, user profile management, booking history, and administrative dashboard views.
  - Application Layer (NestJS Controllers and Services): Handles core business logic including authentication, role-based access control, trip scheduling, seat reservation, booking management, payment processing, cancellation handling, and refund workflows.
  - Data Layer (PostgreSQL with Prisma ORM): Responsible for storing and managing data related to users, buses, routes, trips, seats, bookings, payments, and refunds while maintaining relational integrity and consistency.

- MVC (Model–View–Controller) Pattern
  - Model: Represents entities such as User, Bus, Route, Trip, Booking, Payment, and Refund along with their relationships, constraints, and status management.
  - View: Includes frontend pages and components that display schedules, available seats, booking confirmations, payment information, booking history, and administrative management interfaces.

    - Controller: Receives API requests, validates input data, invokes application services, and returns appropriate responses to the frontend.

- Service Layer Pattern:
    - The Service Layer Pattern encapsulates business logic and operational rules separately from controllers. This pattern is used for functionalities such as seat availability checking, booking confirmation logic, payment status handling, cancellation policy enforcement, and refund eligibility verification. It helps keep controllers lightweight and improves maintainability.
- Repository Pattern:
    - This pattern is implemented through Prisma ORM to centralize database operations and query handling. It manages create, read, update, and delete (CRUD) operations for major modules while maintaining separation between business logic and persistence logic.
- Observer Pattern:
    - This pattern supports status-driven operations within the system. For example, booking status changes after successful payment, seat status updates after cancellation or refund processing, and payment/refund status synchronization across modules. This approach improves consistency and coordination between different system components.

Applying these software design and analysis patterns improves code organization, supports modular development, and increases the reliability, maintainability, and scalability of the Bus Ticket Management System. As a result, the system becomes easier to develop, test, maintain, and extend with future functionalities.

4.1 Activity Diagram

4.1.1 Activity Diagram for User Roles (Authentication + Authorization)

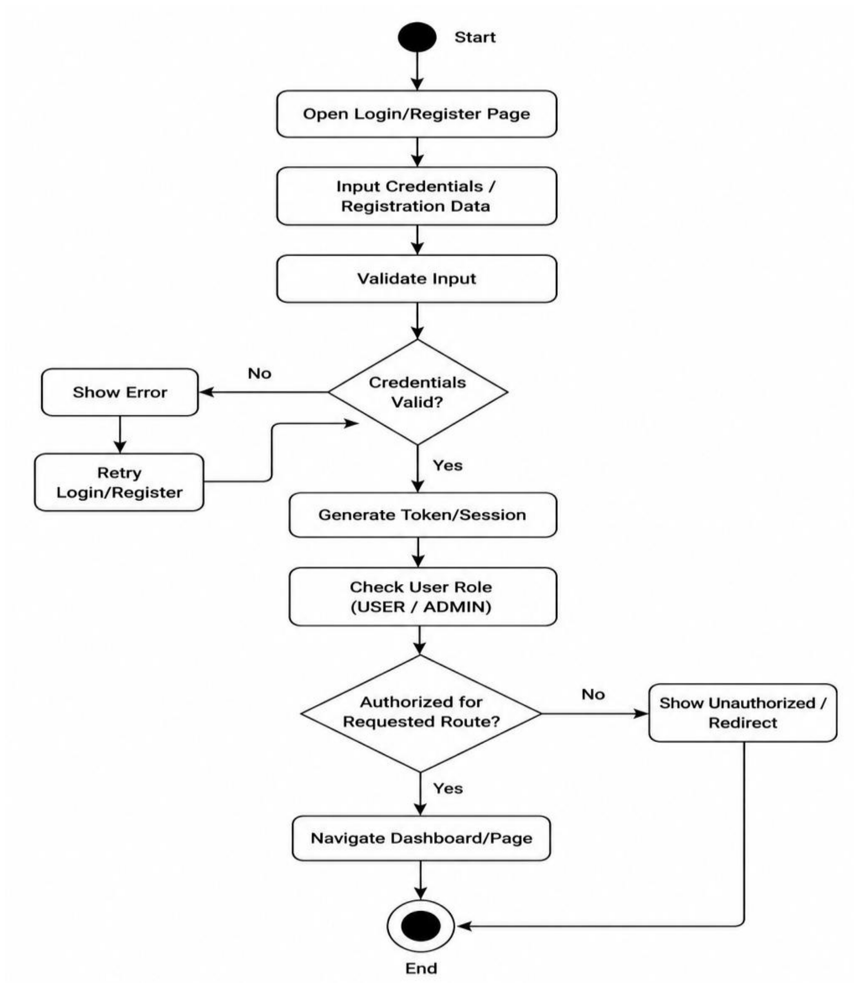


Figure 4.1 Activity Diagram for User Roles.

4.1.2 Activity Diagram for Trip Search

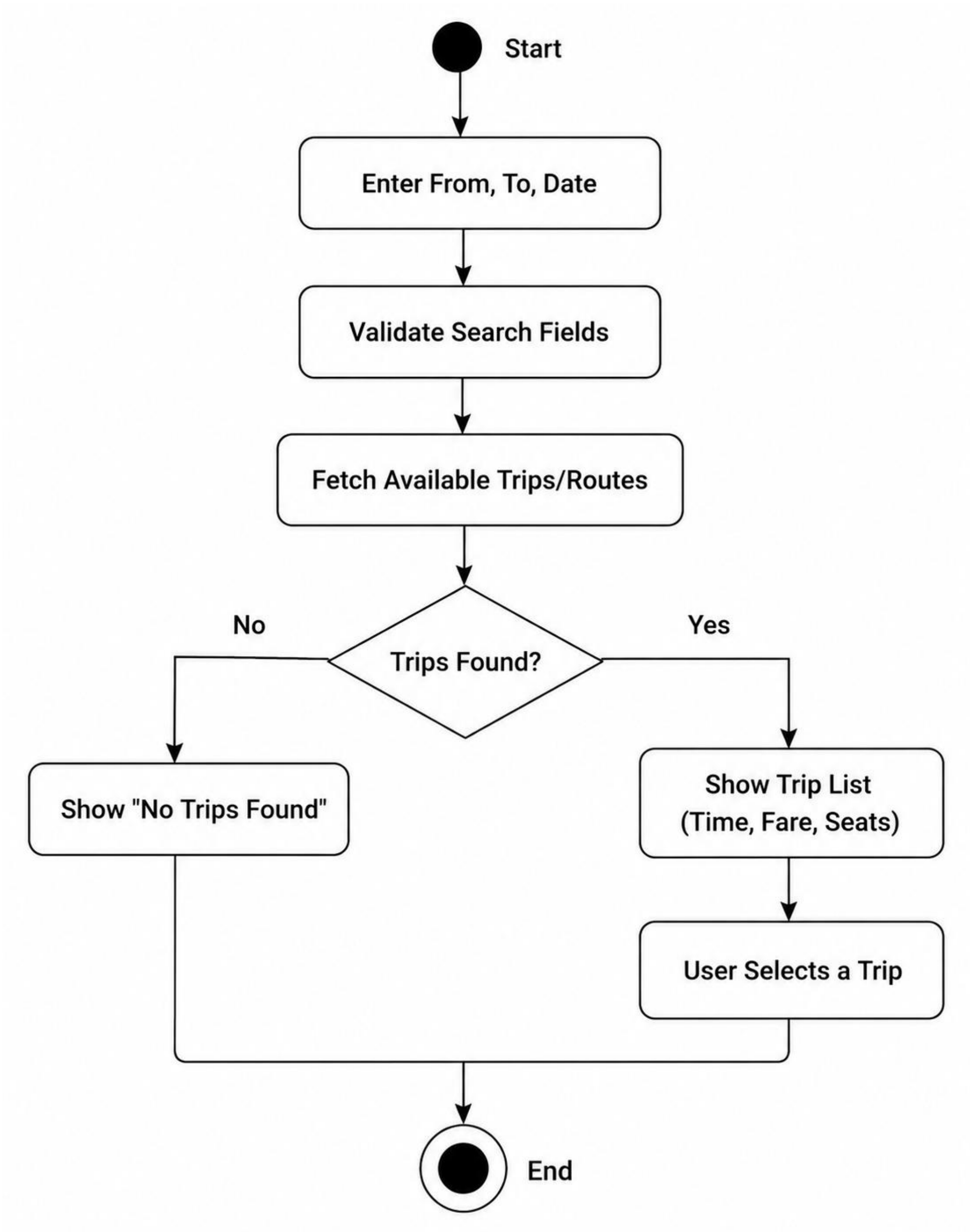


Figure 4.2 Activity Diagram for Trip Search.

4.1.3 Activity Diagram for Seat Selection

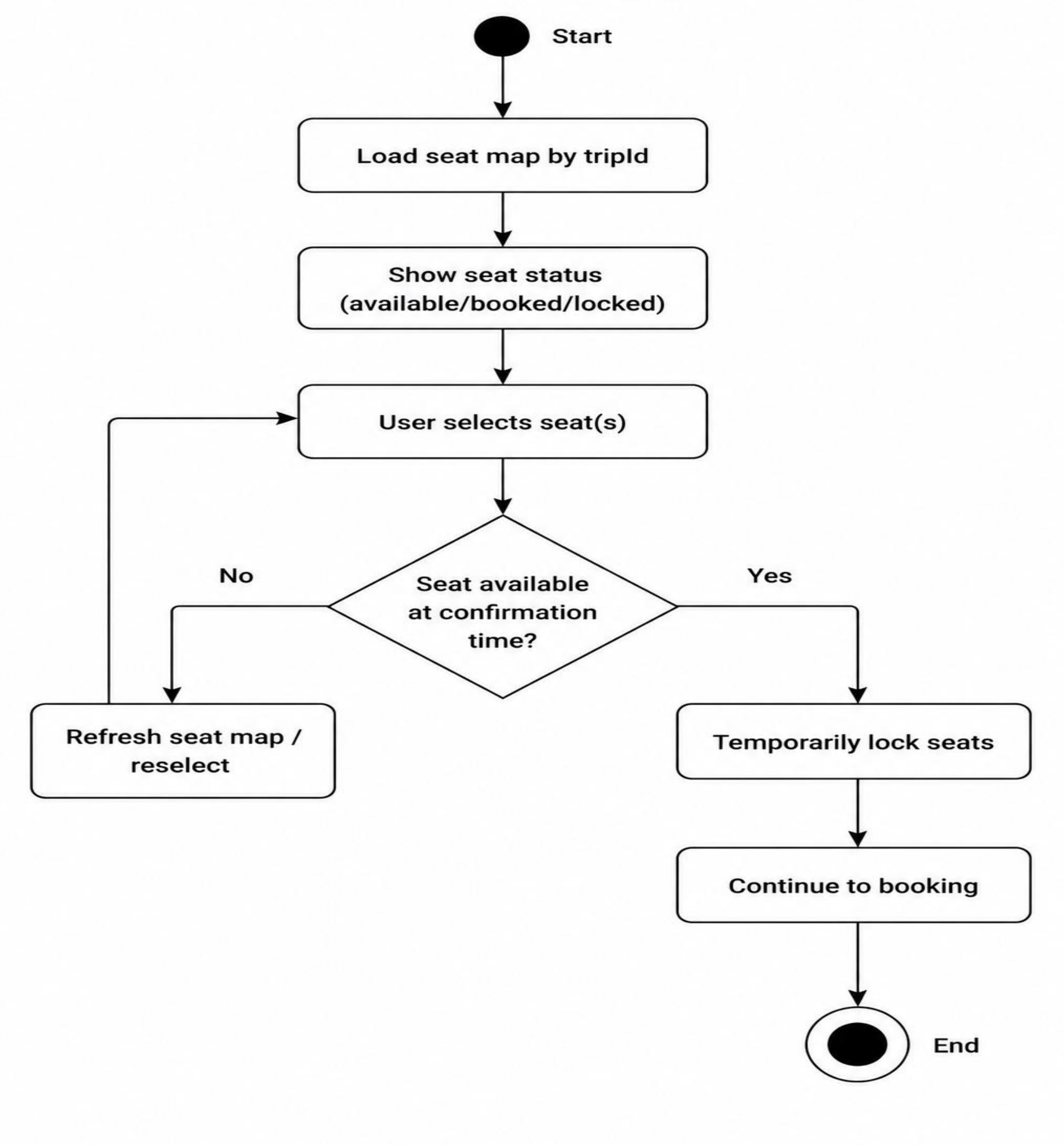


Figure 4.3 Activity Diagram for Seat Selection

4.1.4 Activity Diagram for Booking Summary & Confirmation

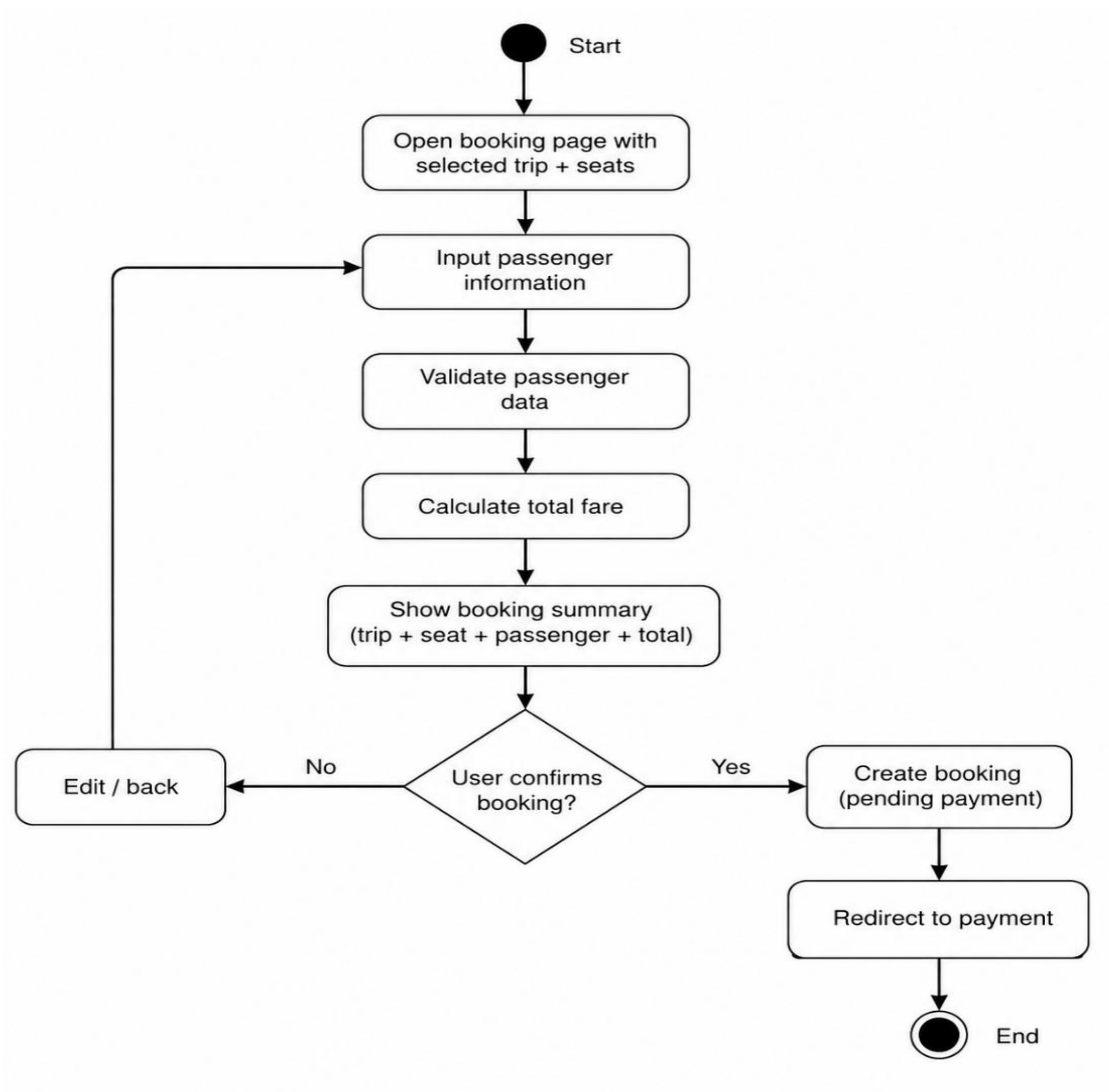


Figure 4.4 Activity Diagram for Booking Summary and Confirmation.

4.1.5 Activity Diagram for Payment Flow

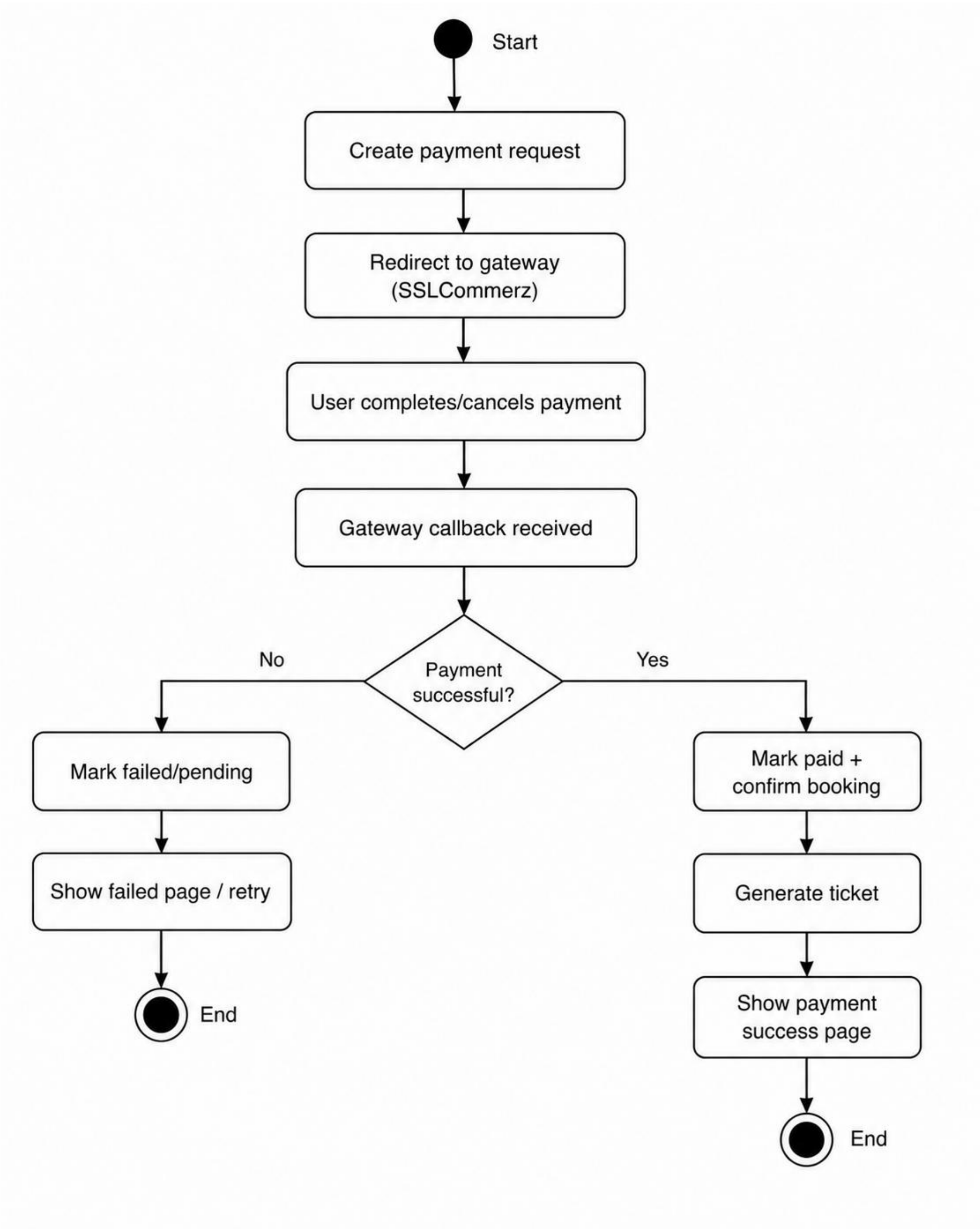


Figure 4.5 Activity Diagram for Payment Flow.

4.1.6 Activity Diagram for Refund Request (user Side)

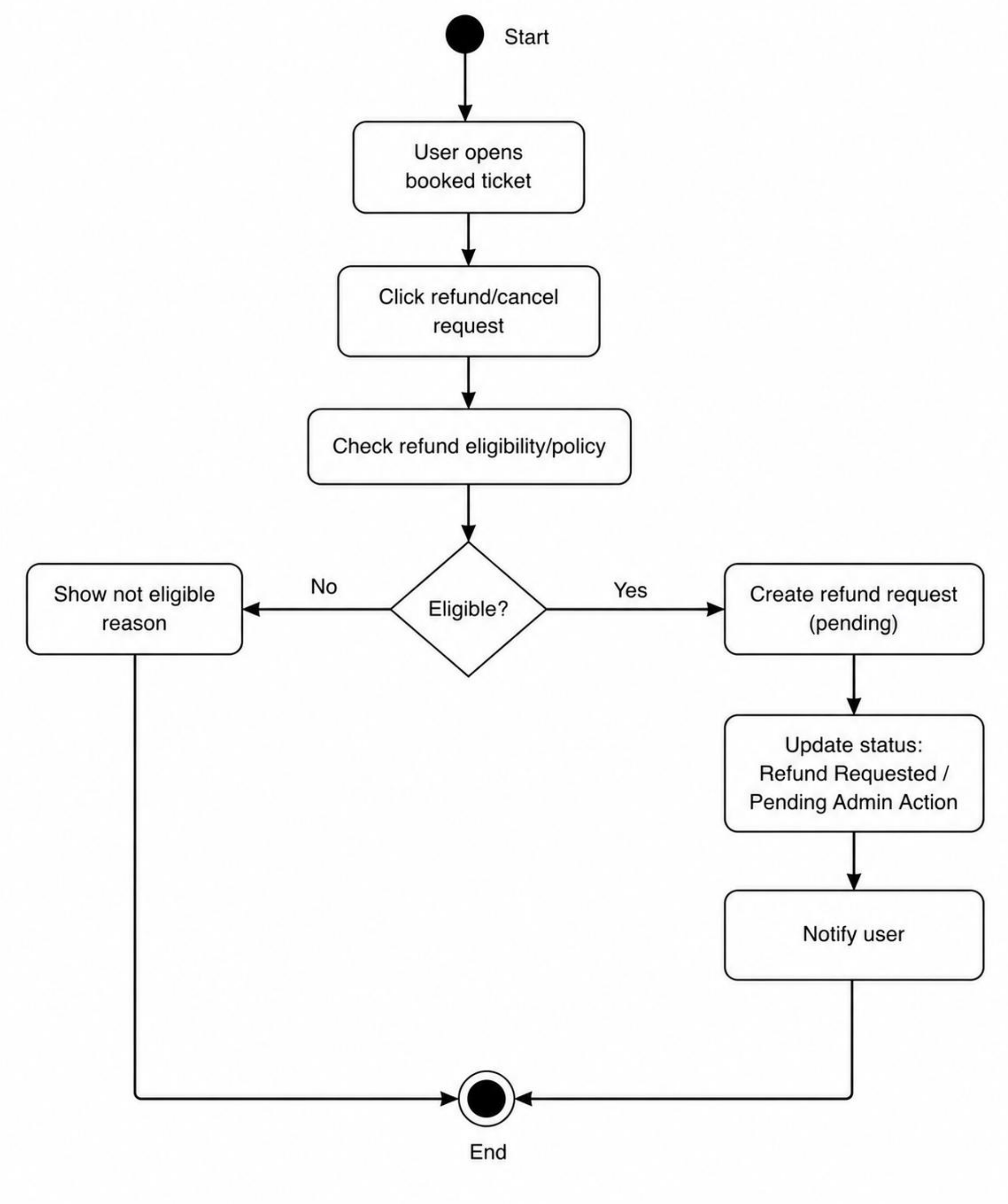


Figure 4.6 Activity Diagram for refund Request (user Side).

4.1.7 Activity Diagram for Refund Request (admin Side)

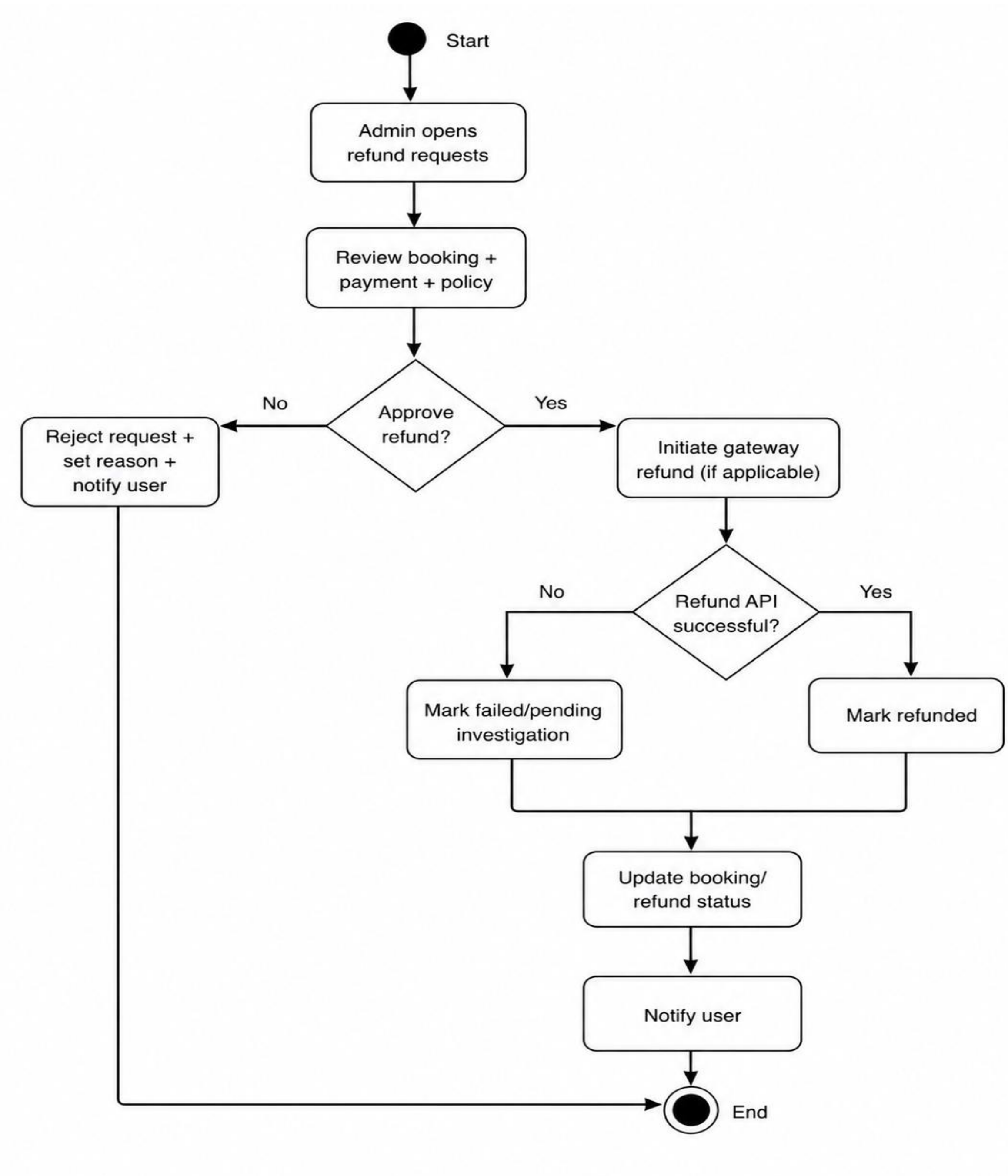


Figure 4.7 Activity Diagram for refund Request (admin Side).

4.1.7 Activity Diagram for Admin

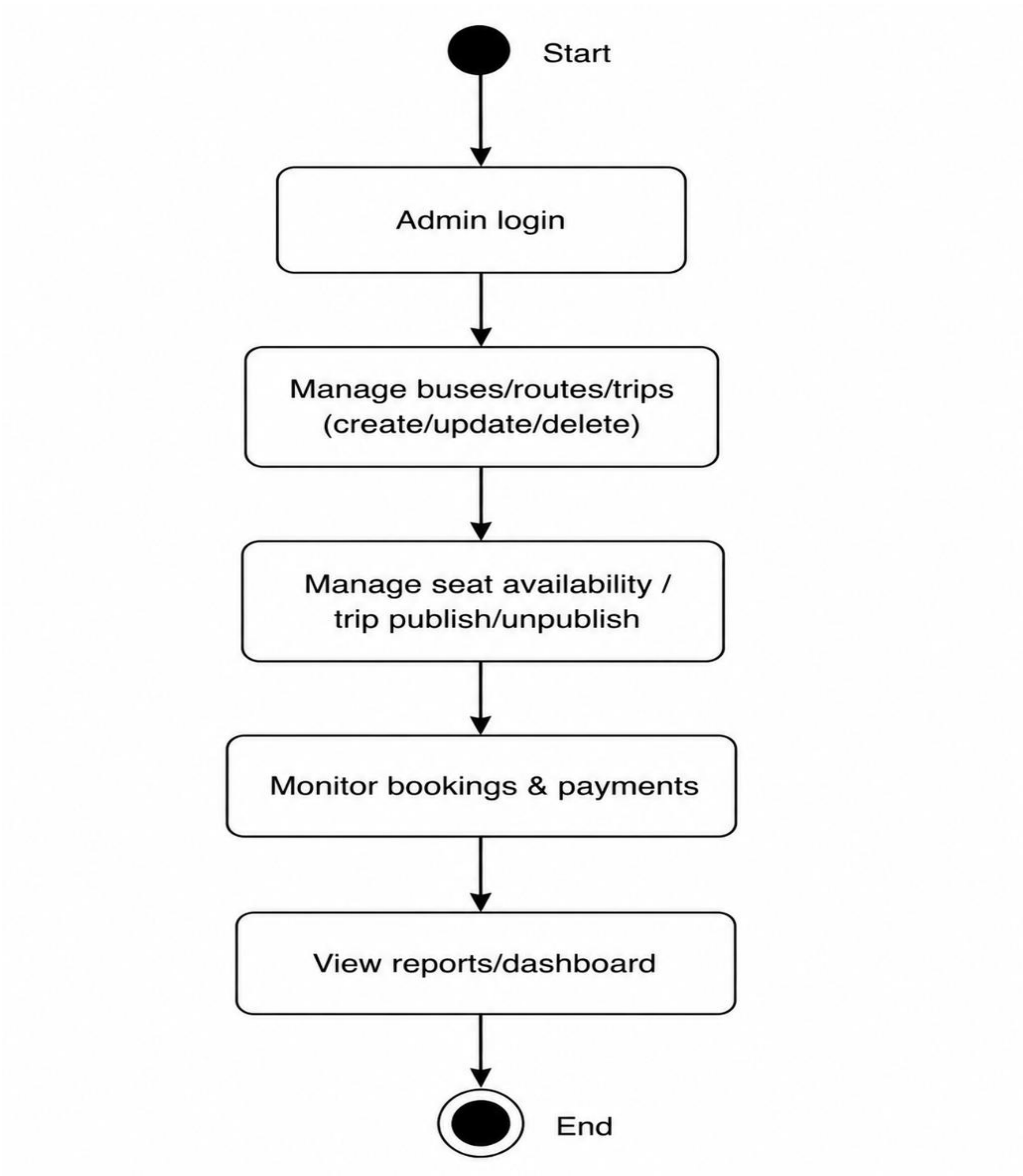


Figure 4.8 Activity Diagram for Admin

Chapter 5.

Project Management

5.1 Risk Identification:

Risk identification is an important step in the risk management process that focuses on identifying potential threats and challenges that may affect the successful development, deployment, and operation of the Bus Ticket Management System. Proper identification of risks helps minimize unexpected failures and supports effective planning, monitoring, and mitigation throughout the project lifecycle.

Several techniques were used to identify potential risks, including brainstorming sessions, observation of similar online ticket booking systems, analysis of project requirements, and discussions regarding operational and technical challenges. The identified risks mainly involve payment processing, booking conflicts, authentication, database operations, system performance, and administrative data management.

Table 2: Risk Identification Table

Risk ID	Risk Description	Potential Impact	Affected Area
R1	SSLCommerz payment session or callback misconfiguration (e.g., incorrect backend URL, failed order validation, weak IPN handling)	Inconsistent booking or payment states; bookings may remain unpaid despite successful payment	Payment Module (NestJS Payment Services and SSLCommerz Integration)
R2	Seat concurrency issues caused by overlapping seat locks or lock expiration conflicts during booking confirmation	Double booking, failed checkout process, and passenger dissatisfaction	Booking Module and Seat Management Workflow
R3	JWT access token expiration during booking or payment operations without proper token refresh handling	Users may be redirected to login during active booking or payment sessions	Authentication Module and Frontend Route Protection
R4	PostgreSQL database failure, Prisma migration issues, or deployment-related database errors	System downtime, failed transactions, or possible data inconsistency	Database Layer (PostgreSQL and Prisma ORM)
R5	Refund processing failures caused by missing payment gateway prerequisites or delayed administrative approvals	Refund requests may remain pending or fail to complete successfully	Refund Management and Payment Gateway Integration
R6	Incorrect bus, route, trip, or pricing information entered by administrators	Passengers may receive incorrect schedules, fares, or booking information	Administrative Management Module

Risk ID	Risk Description	Potential Impact	Affected Area
R7	SMTP or Nodemailer configuration issues affecting OTP verification and email notifications	Users may fail to receive password reset links or booking confirmations	Email and Notification Services
R8	High traffic load affecting trip searching, seat availability, and booking APIs	Slow response time, request failures, or degraded user experience	Trip Search, Seat Management, and Booking APIs

5.2 Risk Analysis:

Risk analysis is the process of evaluating identified risks based on their probability of occurrence and potential impact on the Bus Ticket Management System. This process helps determine the severity and priority of each risk so that appropriate mitigation strategies can be planned effectively.

For this project, risks were analyzed using a qualitative risk assessment approach where probability and impact values were assigned on a scale of 1 to 5. The overall risk score was calculated by multiplying probability and impact values. Based on the calculated scores, risks were categorized into high, medium, and low priority levels.

Table 3: Risk analysis Table

Risk ID	Probability	Impact	Risk Score	Priority
R1	4	4	16	High
R2	3	5	15	High
R3	3	3	9	Medium
R4	2	5	10	Medium
R5	3	4	12	Medium
R6	2	4	8	Medium
R7	3	2	6	Low
R8	4	4	16	High

5.3 Risk Planning

Risk planning involves defining mitigation strategies and preventive actions to minimize the impact of identified risks on the Bus Ticket Management System. Proper planning helps ensure stable system operation, secure transaction handling, and efficient project management throughout development and deployment phases.

Table 4: Risk analysis Table

Risk ID	Mitigation Action	Owner	Timeline
R1	Validate payment callbacks properly, maintain transaction identifiers, and ensure correct backend/frontend configuration for payment verification	Backend Developer	Before staging deployment
R2	Implement proper seat-lock timeout management, cleanup processes, and final seat availability validation during booking confirmation	Backend Developer	Immediate development sprint
R3	Implement refresh-token handling and session protection to reduce forced logout during booking and payment operations	Frontend and Backend Developers	Iteration 2
R4	Maintain database backup procedures, migration checklists, and rollback strategies before deployment updates	Backend Developer	Before each release
R5	Define refund approval workflow, improve refund queue management, and validate refund-related payment information properly	Backend Developer and Admin Team	Before production deployment
R6	Perform regular verification of trip schedules, fares, and operational data entered by administrators	Admin Management Team	Weekly quality assurance process
R7	Configure and monitor SMTP services properly to ensure reliable OTP and email notification delivery	Backend Developer	Environment setup phase
R8	Optimize database queries, indexing, and API performance to handle high traffic and booking requests efficiently	Backend Performance Team	Load testing phase

Through effective risk planning and mitigation strategies, the Bus Ticket Management System aims to reduce operational disruptions, improve system reliability, and maintain secure and efficient transport management services.

5.4 The RMMM Plan

The Risk Mitigation, Monitoring, and Management (RMMM) plan is used to reduce the impact of identified risks and ensure stable operation of the Bus Ticket Management System throughout development and deployment phases. This plan defines preventive actions, contingency strategies, and monitoring activities for handling technical, operational, and security-related risks effectively.

Table 5: RMMM Plan Table

Risk ID	Mitigation	Management / Contingency	Monitoring
R1	Maintain payment verification monitoring and callback validation mechanisms	Manual reconciliation process and communication with payment gateway support if failures occur	Regular monitoring of payment verification anomalies
R2	Track seat-lock timeout and booking completion consistency	Force seat refresh and revalidation if booking conflicts occur	Daily monitoring of booking and seat allocation anomalies
R3	Monitor authentication failures and expired session events	Improve session handling and update token management strategies when required	Weekly review of authentication and JWT-related errors
R4	Maintain database health checks, backup procedures, and migration validation	Restore database backups and reapply migrations carefully if failures occur	Continuous database and server monitoring
R5	Monitor refund request processing and approval timelines	Manual refund processing if automated gateway refund operations fail	Daily monitoring of pending refund requests
R6	Maintain audit logs for administrative updates and operational changes	Roll back incorrect trip, fare, or schedule updates when necessary	Regular review of administrative activity reports
R7	Monitor email service configuration and SMTP connectivity	Provide manual support for password reset and account recovery if email delivery fails	Continuous monitoring of SMTP authentication and delivery failures
R8	Monitor API response times and database query performance under traffic load	Temporarily reduce non-critical operations during high traffic periods	Performance monitoring through API and server response analysis

Through the implementation of this RMMM plan, the Bus Ticket Management System aims to minimize operational disruptions, improve reliability, and maintain secure and efficient transportation management services.

Chapter 6.

Project Planning

6.1 System Project Estimation

System project estimation is the process of predicting the required development resources before full implementation. For EasyTrip, estimation is based on functional size, because the project is user-feature driven and involves multiple transactional workflows (booking, payment, refund, admin operations) with persistent relational data.

The estimation process in this chapter follows these steps:

1. Identify functional components (EI, EO, EQ, ILF, EIF).
2. Classify complexity using standard FP rules.
3. Compute Unadjusted Function Points (UFP).
4. Apply Value Adjustment Factor (VAF) using General System Characteristics.
5. Compute Adjusted Function Points (AFP).
6. Convert AFP to effort and project duration.

The major scope considered in this estimation includes:

- User authentication and role-based authorization (JWT)
- Trip search and seat viewing
- Seat selection and booking workflow
- SSLCommerz payment integration
- Refund request and processing workflow
- Admin dashboard and operational control
- Route, bus, and trip management

Technology stack: **React.js, NestJS, PostgreSQL, Prisma ORM, JWT**.

6.2 Function-Oriented Metrics

Function-Oriented Metrics estimate project size based on user-visible functionality rather than lines of code. The five standard function types are used: **EI, EO, EQ, ILF, EIF**.

Table 6: Standard Function Point Weight Table

Function Type	**Low**	**Average**	**High**
External Inputs (EI)	3	4	6
External Outputs (EO)	4	5	7
External Inquiries (EQ)	3	4	6
Internal Logical Files (ILF)	7	10	15
External Interface Files (EIF)	5	7	10

External Inputs (EI)

External Inputs are transactions that capture incoming data and update internal system files. In EasyTrip, representative EIs include:

- Register user
- Login and token refresh
- Update profile / change password
- Create booking / confirm booking / cancel booking
- Start payment
- Request refund
- Admin create/update/cancel trip

- Admin manage route/bus/seat configuration

External Outputs (EO)

External Outputs produce processed or derived data for the user. In EasyTrip, EOs include:

- Admin dashboard metrics (aggregated statistics)
- Payment callback result processing and redirect outcome
- Booking confirmation response with computed status and totals
- Refund decision output (approved/rejected with computed effects)

External Inquiries (EQ)

External Inquiries are retrieval transactions with minimal processing:

- Trip search results
- Trip details with seat availability
- My bookings list
- Booking detail view
- Route/bus/seat list
- Admin filtered booking view
- User list/query view

Internal Logical Files (ILF)

ILFs are logically related data groups maintained by the system. For EasyTrip:

- User
- Bus
- Route
- Trip
- Seat

- Booking
- BookingSeat
- Payment
- Refund

External Interface Files (EIF)

EIFs are external data sources referenced but not maintained internally. For EasyTrip:

- SSLCommerz gateway interaction data (transaction verification/callback payload domain)

6.3 Identifying Complexity

Complexity classification is based on standard Function Point rules:

- **Transaction functions (EI/EO/EQ):** depend on **DET** (Data Element Types) and **FTR** (File Types Referenced).
- **Data functions (ILF/EIF):** depend on **DET** and **RET** (Record Element Types).

The following subsections provide both rule tables and applied project-specific classification.

6.3.1 Identifying Complexity of Transition Function

Complexity Rules Table (EI / EO / EQ)

Table 7: EI Complexity Rules (DET × FTR)

FTR \ DET	**1–4**	**5–15**	**16+**
0–1	Low	Low	Average
2	Low	Average	High
3+	Average	High	High

Table 8: EO Complexity Rules (DET × FTR)

FTR \ DET	**1–5**	**6–19**	**20+**
0–1	Low	Low	Average
2–3	Low	Average	High
4+	Average	High	High

Table 9: EQ Complexity Rules (DET × FTR)

FTR \ DET	**1–5**	**6–19**	**20+**
0–1	Low	Low	Average
2–3	Low	Average	High
4+	Average	High	High

Table 10: DET and FTR Based Complexity Analysis Table (EasyTrip Transactions)

Transaction	Type	Estimated DET	Estimated FTR	Complexity
User Registration	EI	10	1	Low
User Login	EI	6	1	Low
Password Reset	EI	7	1	Low
Profile Update	EI	8	1	Low
Create Booking (Seat Lock)	EI	14	3	High
Confirm Booking	EI	9	2	Average
Cancel Booking	EI	8	3	High
Start Payment Session	EI	10	3	High
Request Refund	EI	7	2	Average
Admin Create Bus/Route/Trip	EI	12	2	Average
Admin Update/Cancel Trip	EI	8	2	Average
Trip Search Query	EQ	8	2	Average
Trip Detail with Seats	EQ	16	3	Average
My Bookings List	EQ	12	2	Average
Booking Detail	EQ	10	2	Average
Route List	EQ	4	1	Low
Bus List / Bus Detail	EQ	5	1	Low
Seat List by Bus/Trip	EQ	6	1	Low
Admin Booking Query	EQ	15	3	High
Booking Confirmation Output	EO	12	2	Average
Payment Callback Response	EO	16	3	Average
Refund Processing Outcome	EO	10	2	Average
Admin Dashboard Aggregates	EO	22	4	High
Trip Generation Summary	EO	8	2	Average

6.3.2 Identifying Complexity of Data Function

Table 11: ILF & EIF Complexity Rules

RET \ DET	1–19 DET	20–50 DET	51+ DET
1 RET	Low	Low	Average
2–5 RET	Low	Average	High
6+ RET	Average	High	High

Table 12: RET and DET Based Complexity Analysis Table (EasyTrip Data Groups)

Data Function	Type	Estimated RET	Estimated DET	Complexity
User	ILF	2	18	Low
Bus	ILF	2	14	Low
Route	ILF	1	8	Low
Trip	ILF	3	20	Average
Seat	ILF	2	10	Low
Booking	ILF	4	24	Average
BookingSeat	ILF	3	16	Low
Payment	ILF	3	18	Low
Refund	ILF	3	20	Average
SSLCommerz Interface Payload Domain	EIF	2	12	Low

6.3.3 Unadjusted Function Point Contribution (Transaction Function)

Based on the transaction-function analysis, the selected counts are:

- EI: Low = 5, Average = 7, High = 3
- EO: Low = 1, Average = 4, High = 1
- EQ: Low = 4, Average = 4, High = 1

Table 13: External Input (EI) Contribution

Complexity	**Count**	**Weight**	**Contribution**
Low	5	3	15
Average	7	4	28
High	3	6	18
Total EI UFP			**61**

Table 14: External Output (EO) Contribution

Complexity	**Count**	**Weight**	**Contribution**
Low	1	4	4
Average	4	5	20
High	1	7	7
Total EO UFP			**31**

Table 15: External Inquiry (EQ) Contribution

Complexity	Count	Weight	Contribution
Low	4	3	12
Average	4	4	16
High	1	6	6
Total EQ UFP			**34**

Table 16: Total Transaction Function UFP

Function Type	UFP
EI UFP	61
EO UFP	31
EQ UFP	34
Total	**126**

6.3.4 Unadjusted Function Point Contribution (Data Function)

From the data-function analysis, the selected counts are:

- ILF: Low = 6, Average = 3, High = 0
- EIF: Low = 1, Average = 0, High = 0

Table 17: Internal Logical File (ILF) Contribution

Complexity	Count	Weight	Contribution
Low	6	7	42
Average	3	10	30
High	0	15	0
Total ILF UFP			**72**

Table 18: External Interface File (EIF) Contribution

Complexity	Count	Weight	Contribution
Low	1	5	5
Average	0	7	0
High	0	10	0
Total EIF UFP			**5**

Table 19: Total Data Function UFP

Function Type	UFP
ILF UFP	72
EIF UFP	5
Total	**77**

6.3.5 Performance and Environmental Impact

To adjust UFP for technical/environmental realities, the 14 General System Characteristics (GSC) are rated from 0 to 5.

Table 20: General System Characteristics

#	**General System Characteristic**	**Degree of Influence (0–5)**
1	Data communications	3
2	Distributed data processing	2
3	Performance objectives	2
4	Heavily used configuration	1
5	Transaction rate	2
6	Online data entry	3
7	End-user efficiency	2
8	Online update	2
9	Complex processing	2
10	Reusability	1
11	Installation ease	1
12	Operational ease	1
13	Multiple sites	1
14	Facilitate change	1
Total Degree of Influence (TDI)		**24**

Total Degree of Influence (TDI)

TDI=3+2+2+1+2+3+2+2+2+1+1+1+1+1 = 24

6.3.6 Counting Adjusted Function Point

Step 1: Calculate UFP

UFP=Total UFP of Transition Function + Total UFP of Data Function

$UFP=126+77=203$

Step 2: Calculate VAF

$VAF=0.65+(0.01\times TDI)$

VAF=0.65+(0.01×24)

=0.89

Step 3: Calculate AFP

AFP=UFP×VAF

$AFP=UFP\times VAF$

AFP=203×0.89

=180.67≈181

Therefore, the adjusted functional size of EasyTrip is approximately **181 Function Points**.

6.4 Effort Estimation

The effort estimation for the proposed system is calculated using the productivity rate of 6.5 person-hours per Function Point (FP).

Adjusted Function Points (AFP) = 181

Productivity Rate = 6.5 person-hours / FP

Total Effort (Hours) = AFP × Productivity Rate

$=181 \times 6.5$

$=1176.5$ person-hours

Assuming 1 person-day consists of 8 working hours:

Total Effort (Person-Days) = Total Effort (Hours) / 8

$=1176.5/8$

$=147.06 \approx 147$ person-days

Assuming a development team of 2 members:

Work Per Person = 147/2

$=73.5$ person-days

Assuming 24 working days per month:

Project Duration = 73.5/24

$= 3.061 \approx 3$ months

Therefore, the estimated development effort for the EasyTrip Bus Ticket System is approximately 1176.5 person-hours or 147 person-days. With a team of 2 members, the estimated project completion time is approximately 3 months.

6.5 Project Schedule Chart

Table 21: 12-Week Project Schedule

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Requirement Analysis	✓	✓										
System Design		✓	✓									
Database Design		✓	✓									
Backend Development			✓	✓	✓	✓	✓	✓				
Frontend Development				✓	✓	✓	✓	✓				
Booking Module					✓	✓	✓					
Payment Integration						✓	✓	✓				
Refund Module							✓	✓	✓			
Testing								✓	✓	✓	✓	
Deployment										✓	✓	
Documentation	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

6.6 Account Table

Table: 22 Accounts Table

Cost Category	Description	Estimated Cost (BDT)
Personnel Cost	Developer and project-related workforce expenses	60,000
Expected Hardware Cost	Laptop, internet, and related hardware support	15,000
Expected Software Cost	Development tools, hosting, APIs, and software services	10,000
Expected Other Cost	Documentation, communication, testing, and miscellaneous expenses	5,000
Total Estimated		**90,000 BDT**

Chapter 7.

Design

7.1 Interface Design

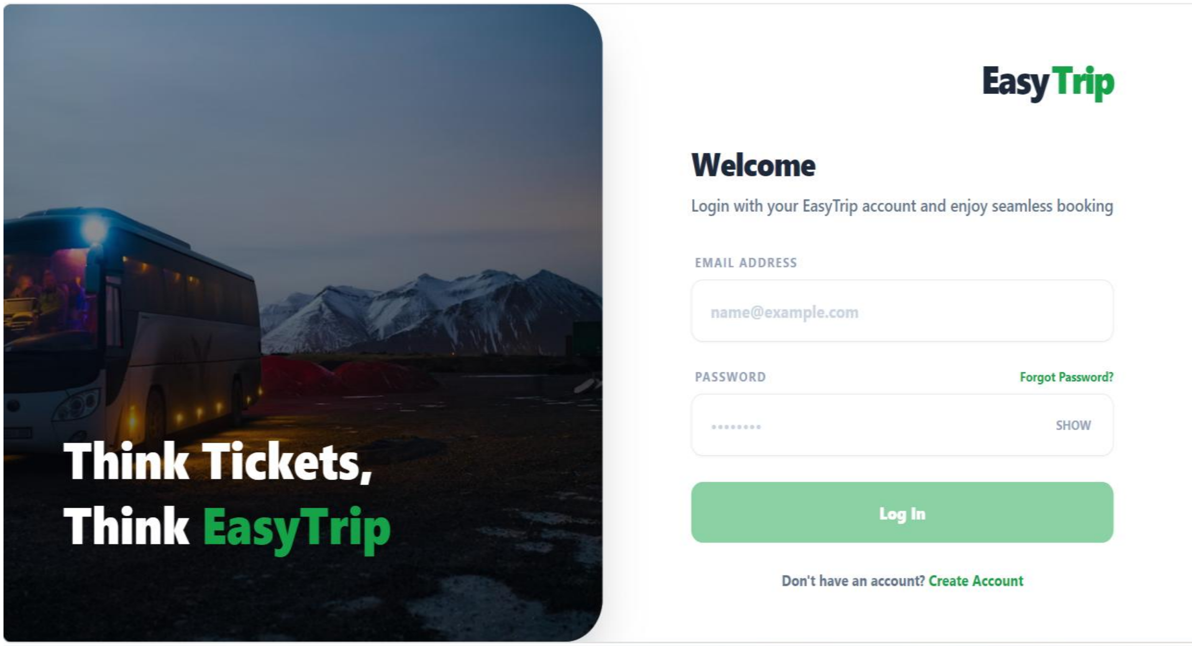


Figure 7.1 Log In Page

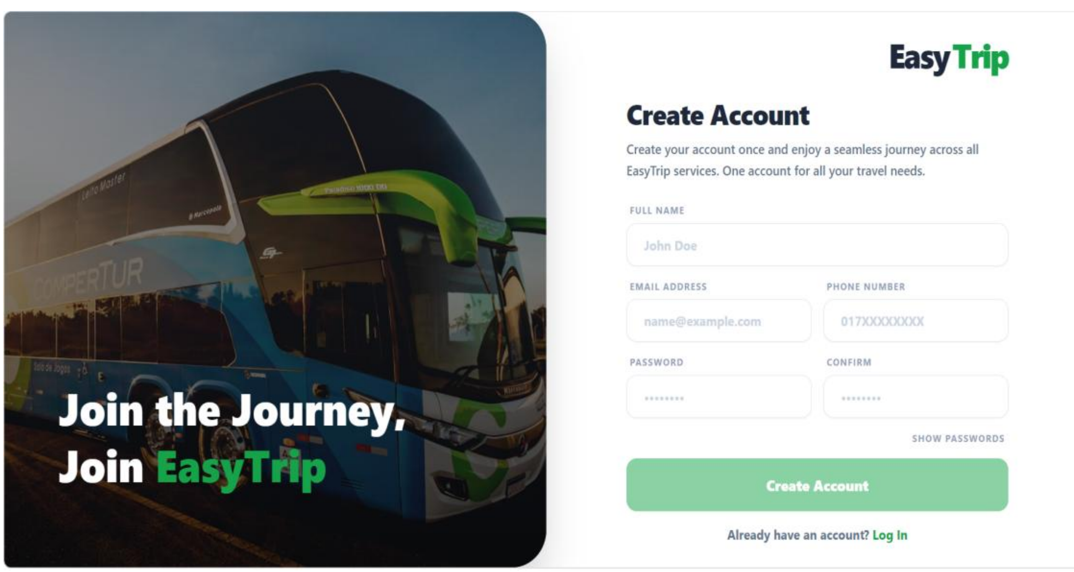


Figure 7.2 Registration Page

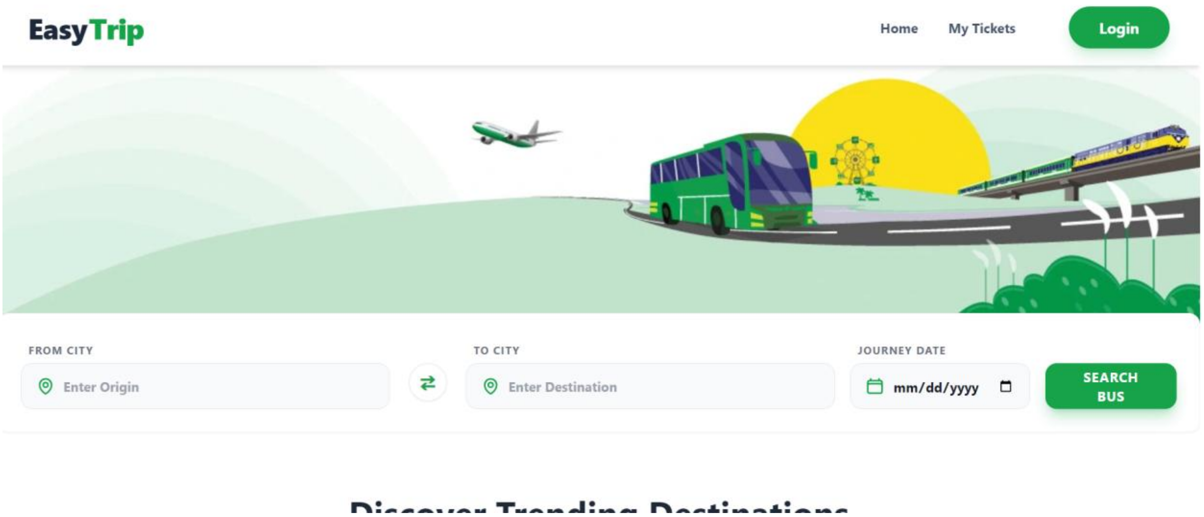


Figure 7.3 Home Page

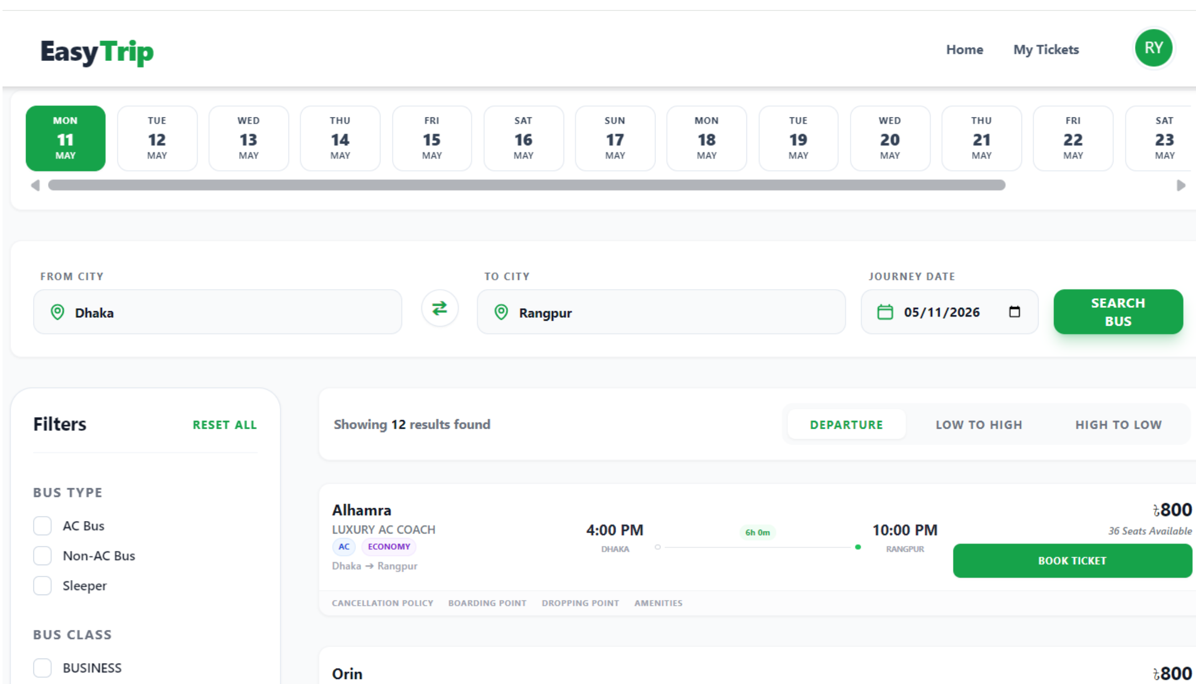


Figure 7.4 Search Trip Page

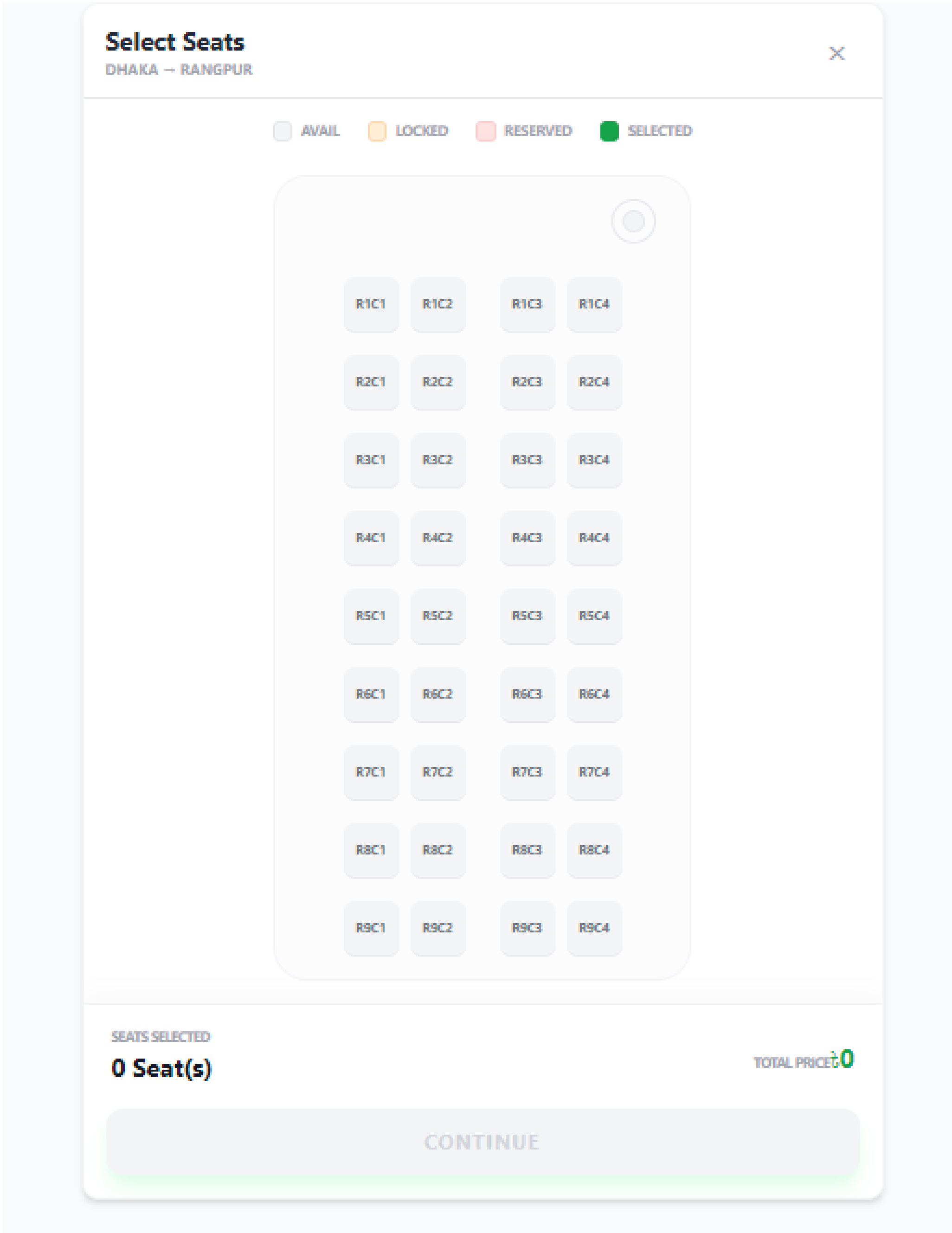


Figure 7.5 Seat layout Page (Economy Class)

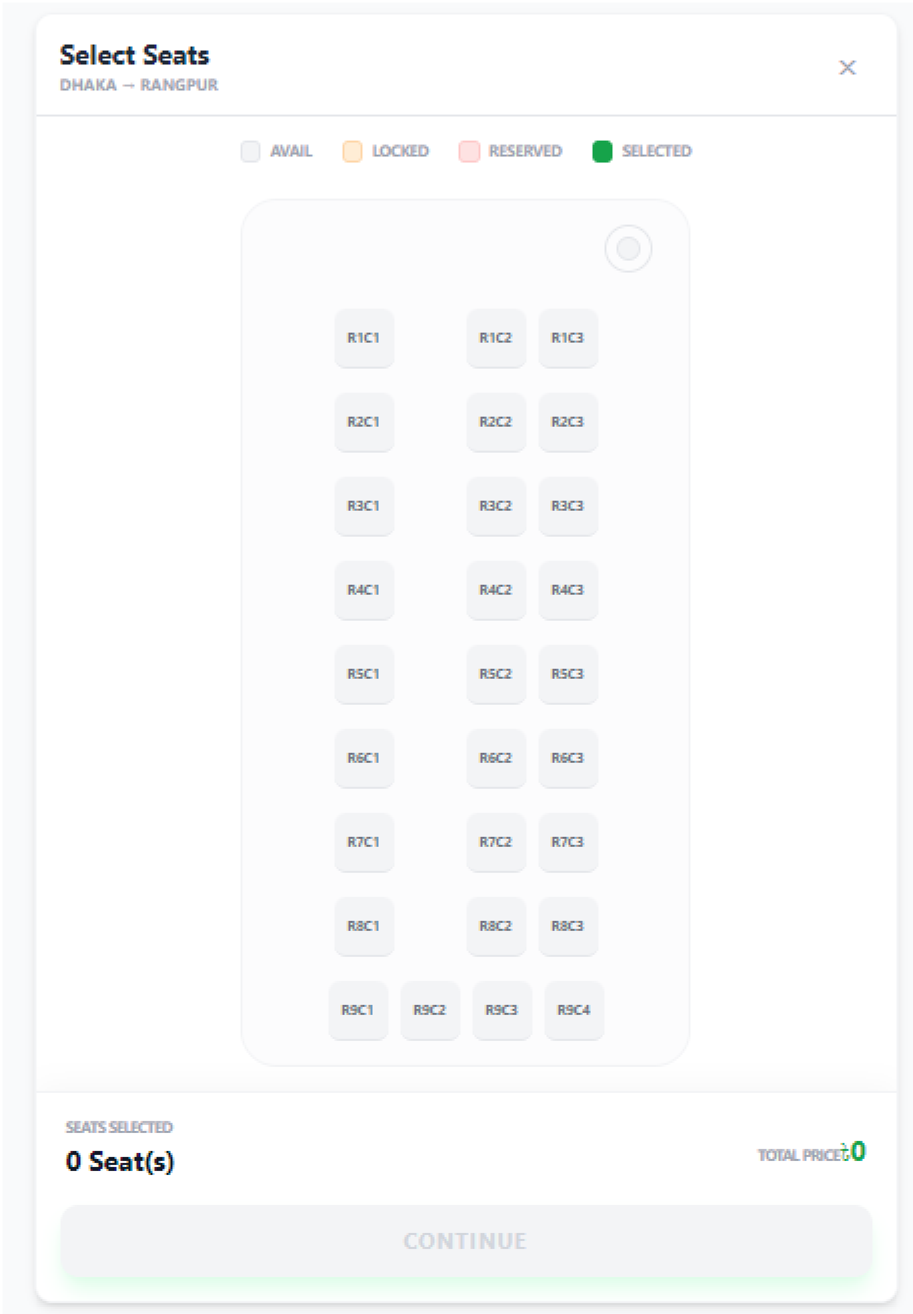


Figure 7.6 Seat Layout Page (Business Class)

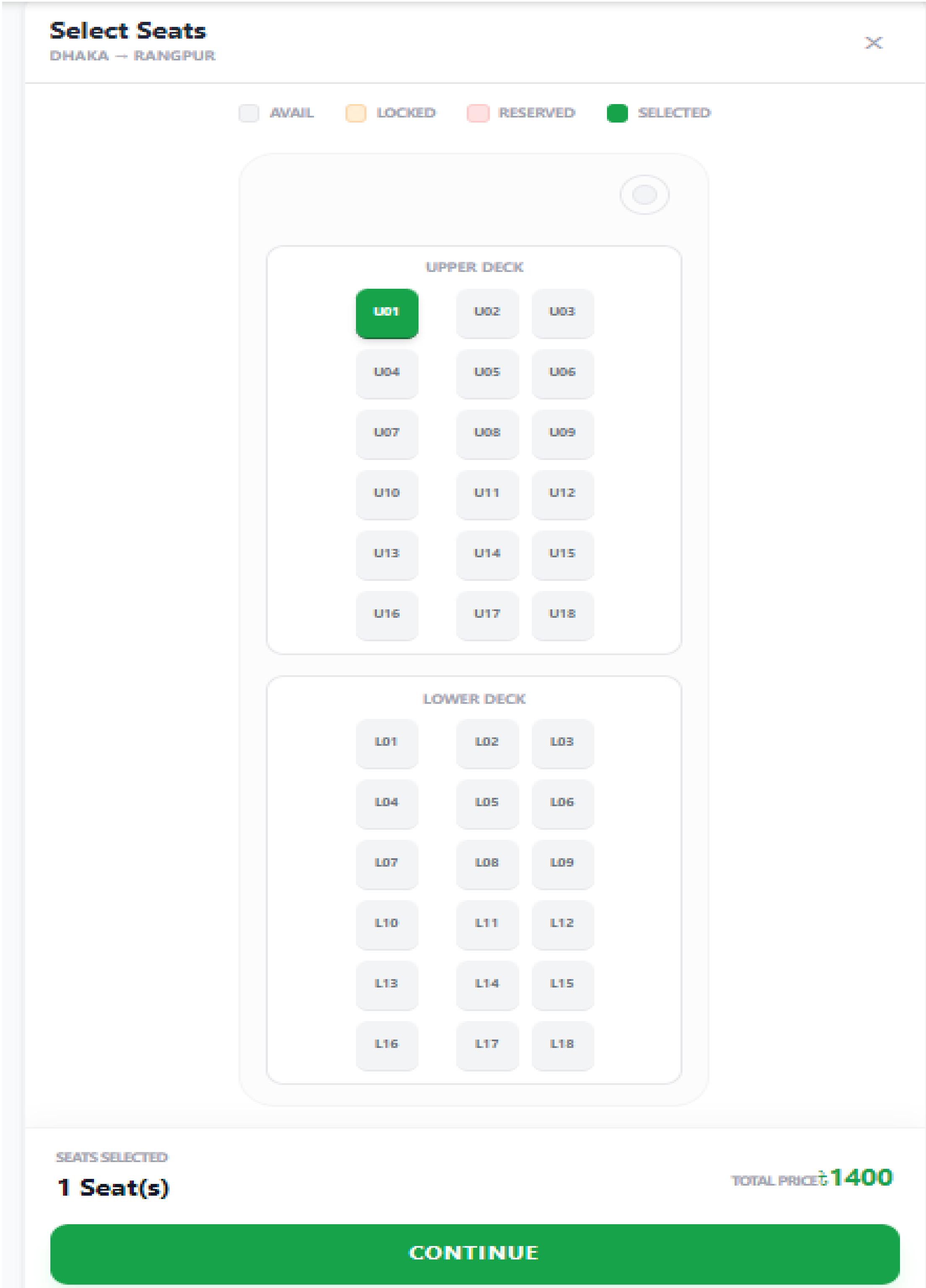


Figure 7.7 Seat Layout Page (Sleeper)

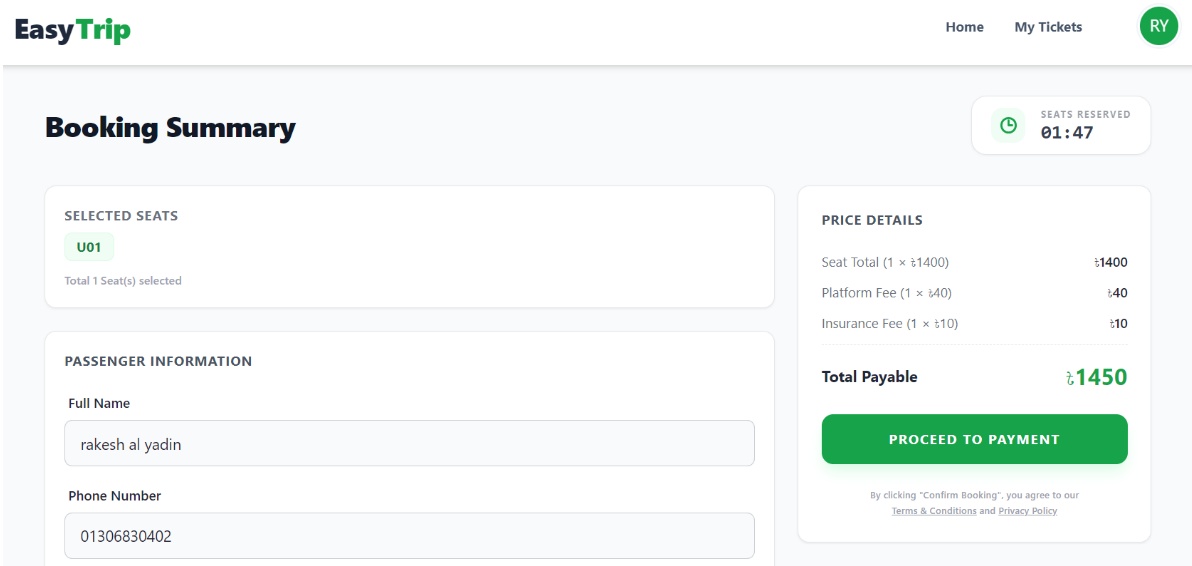


Figure 7.8 Booking Summary Page

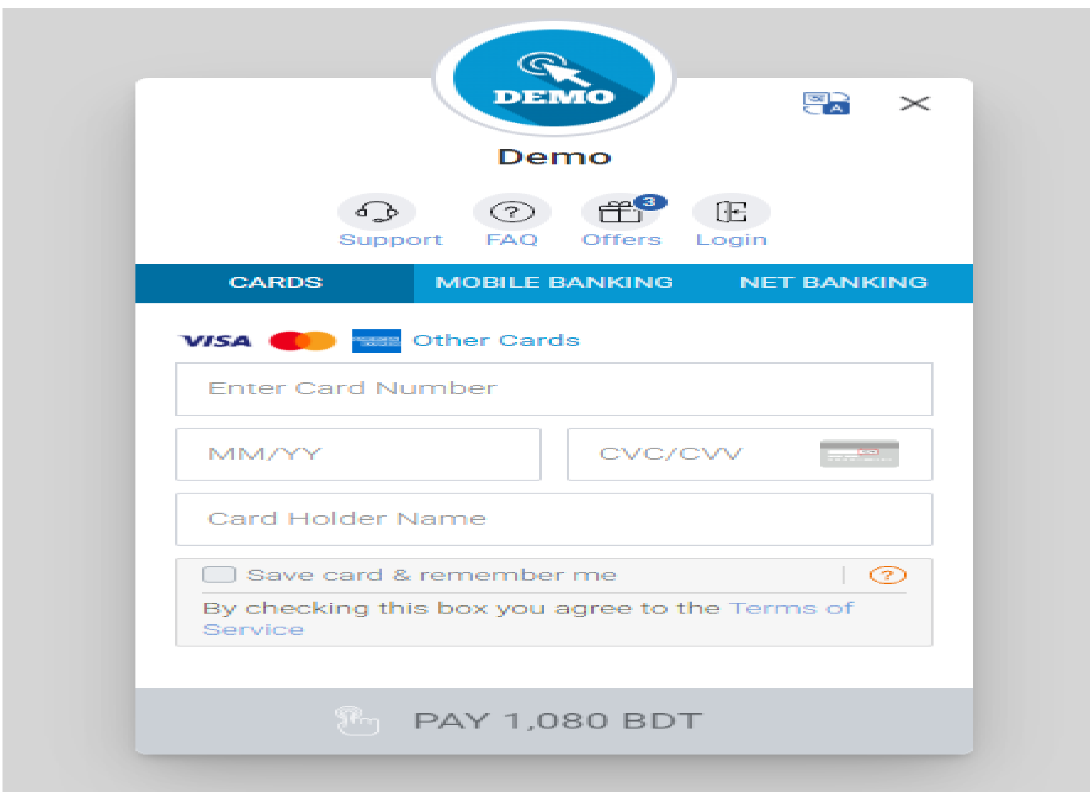


Figure 7.9 Payment Gateway Page

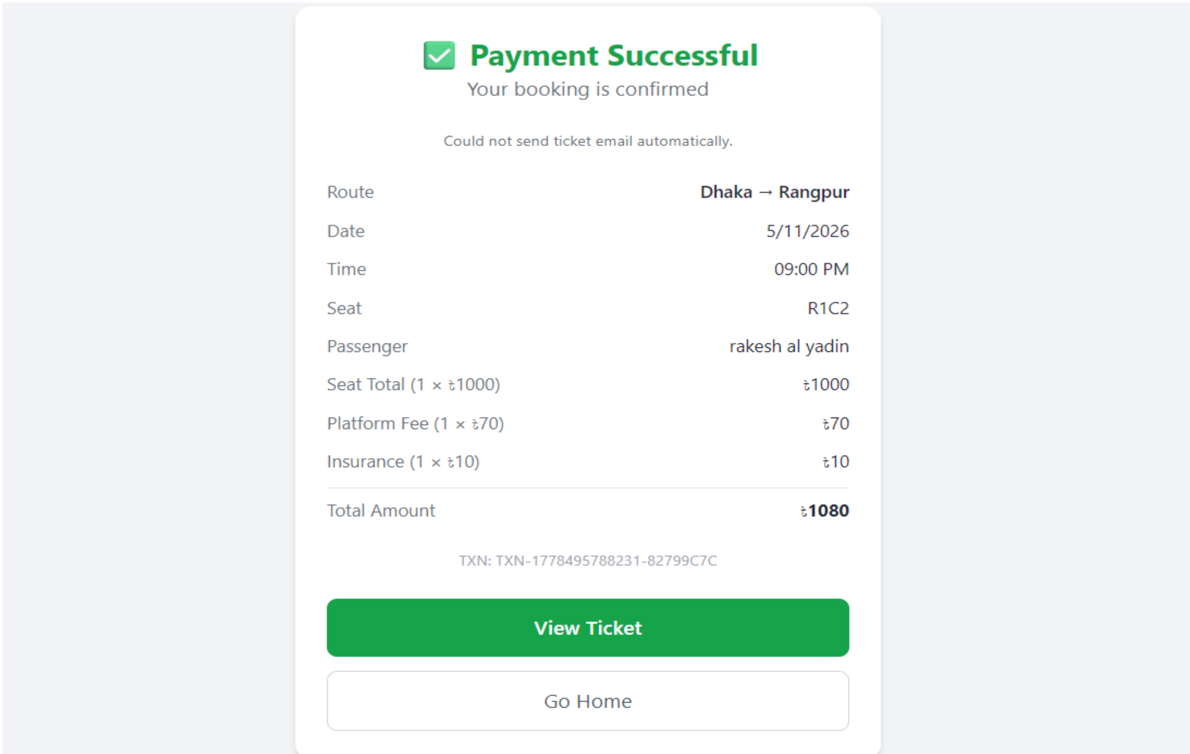


Figure 7.10 Ticket Confirmation Page

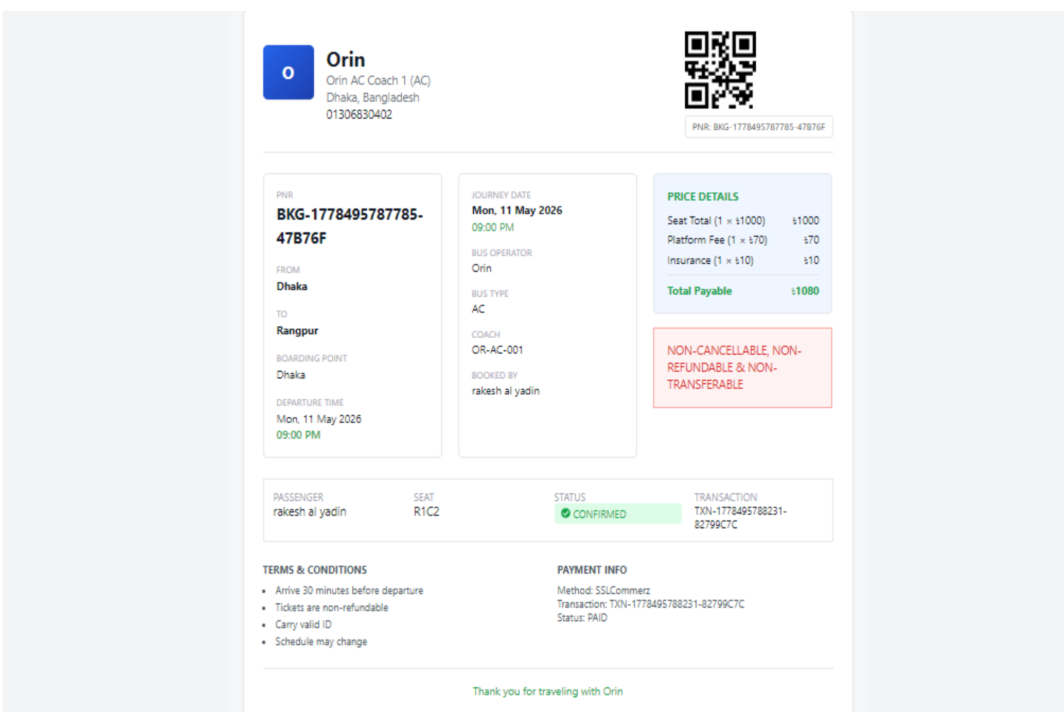


Figure 7.11 Ticket Page

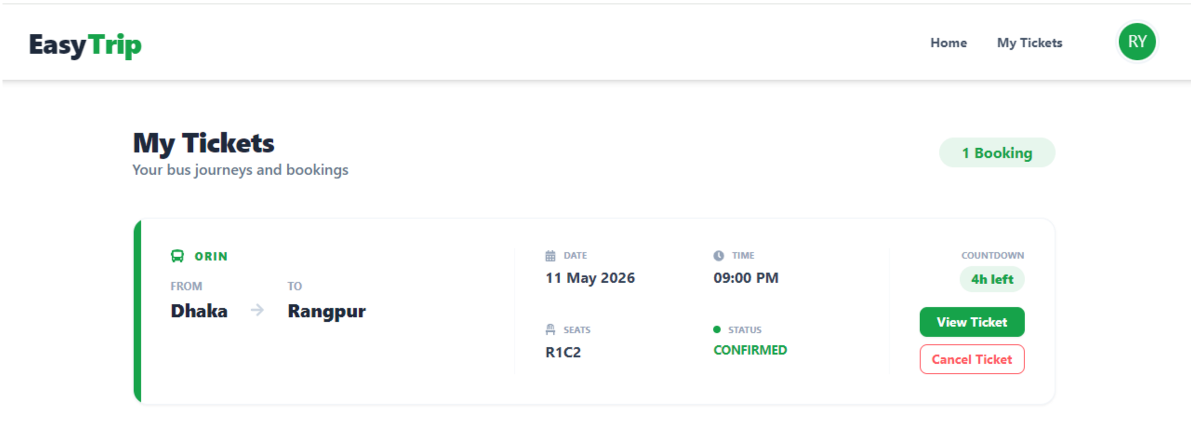


Figure 7.12 My Ticket Page

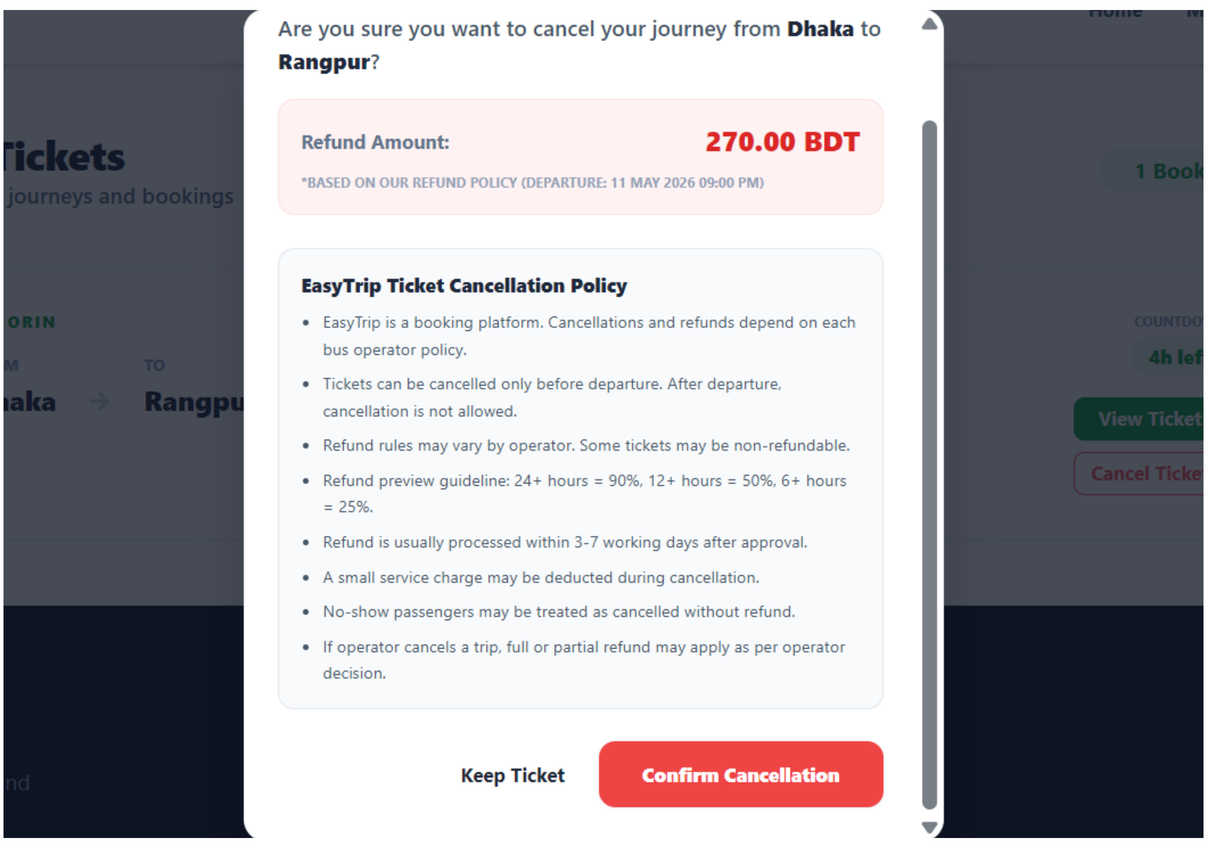


Figure 7.13 Ticket Cancelation page Page

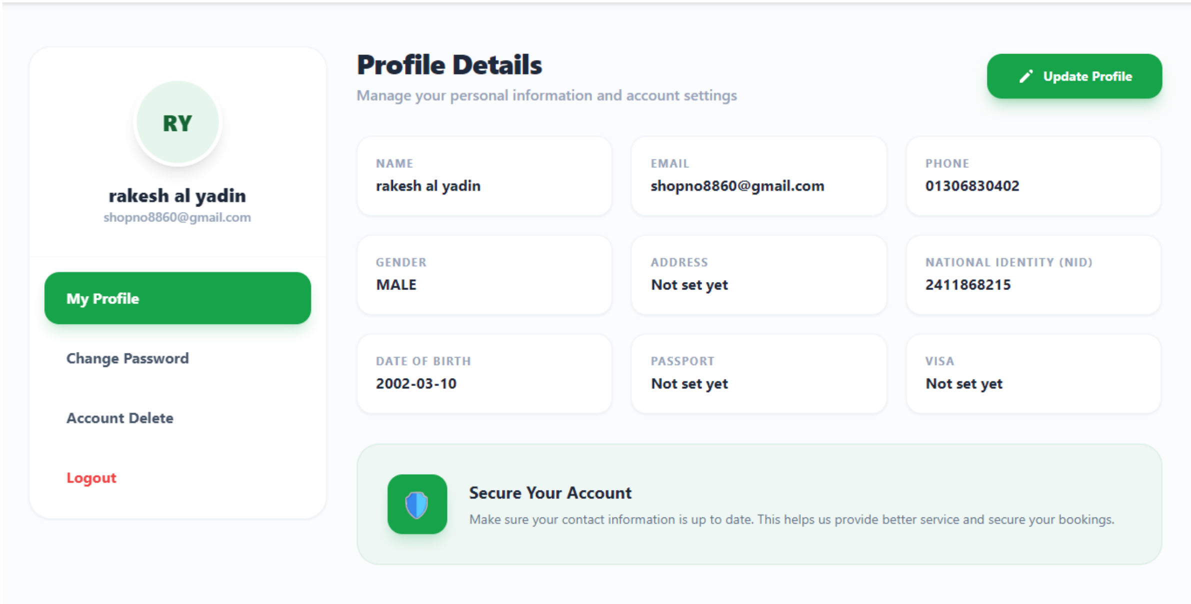


Figure 7.14 My Profile Page

Figure 7.15 Navigation Bar

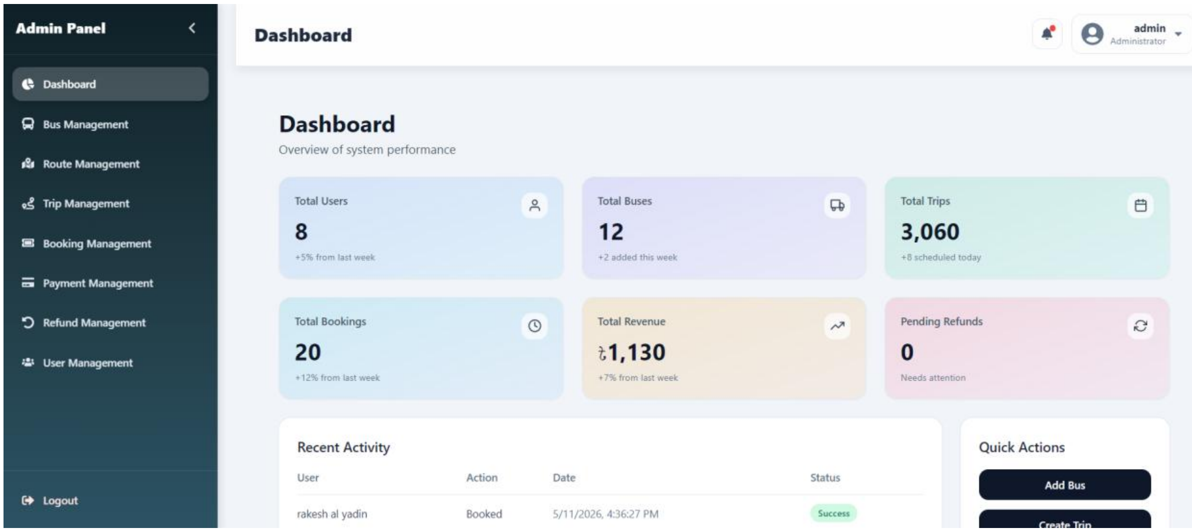


Figure 7.16 Admin Dashboard Page

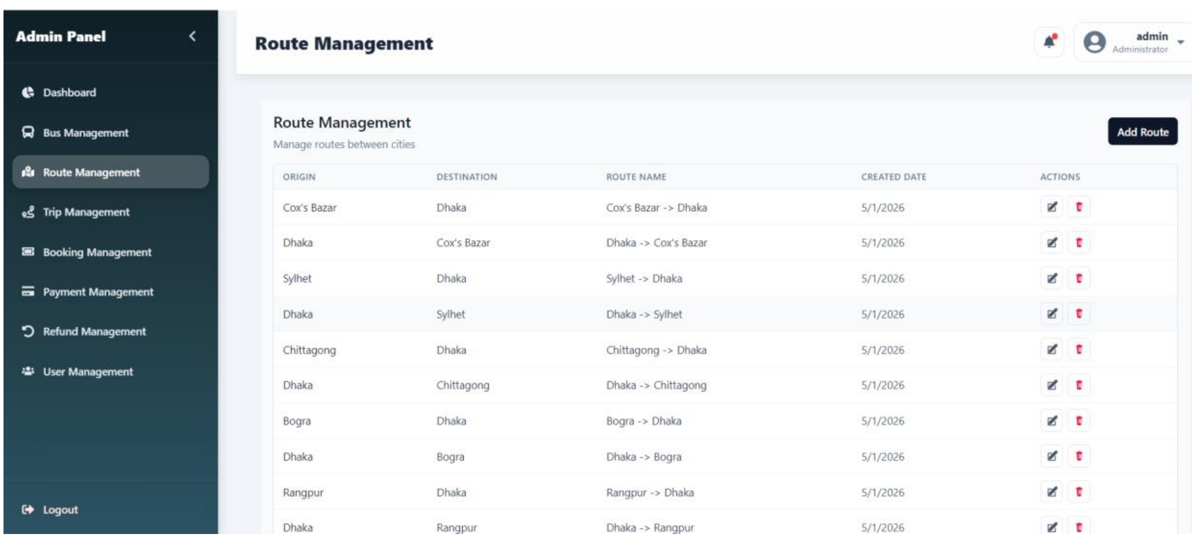


Figure 7.17 Route Management Page

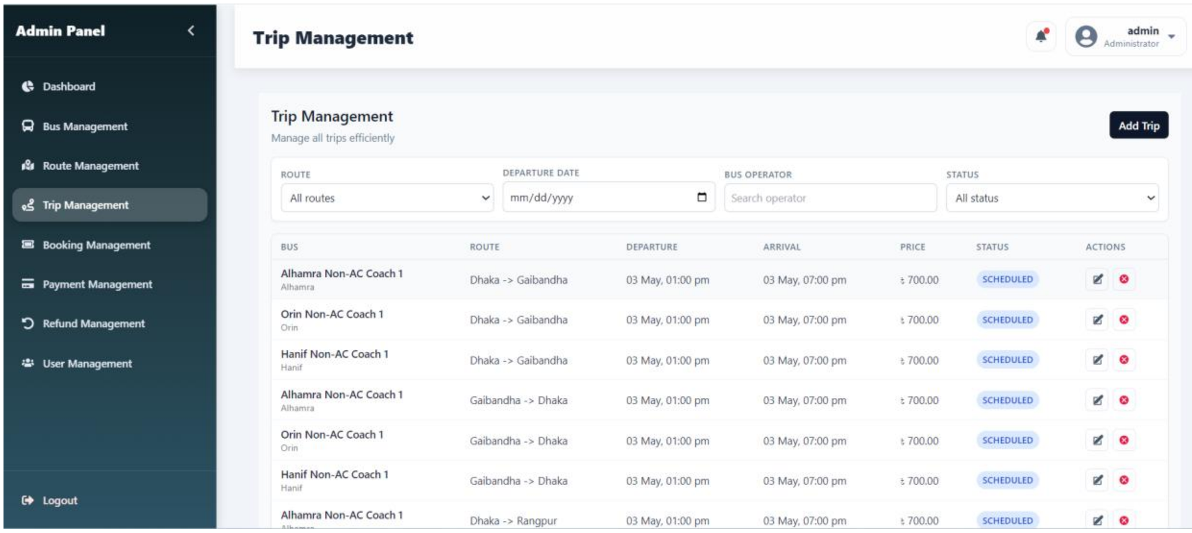


Figure 7.18 Trip Management Page

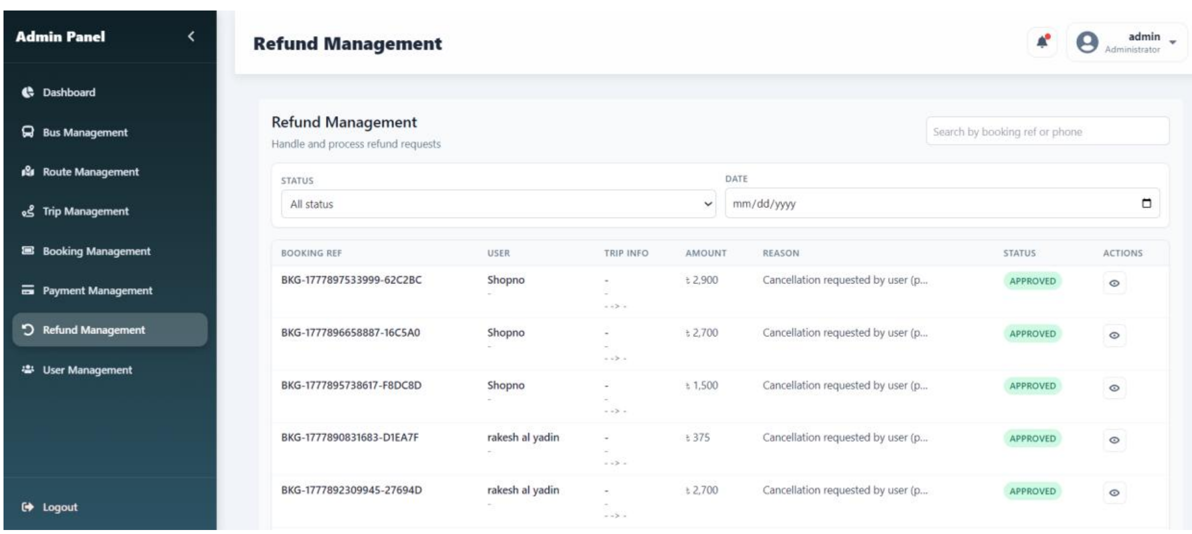


Figure 7.19 Refund Management Page

7.2 Data Flow Diagram

7.2.1 Context Level (Level-0)

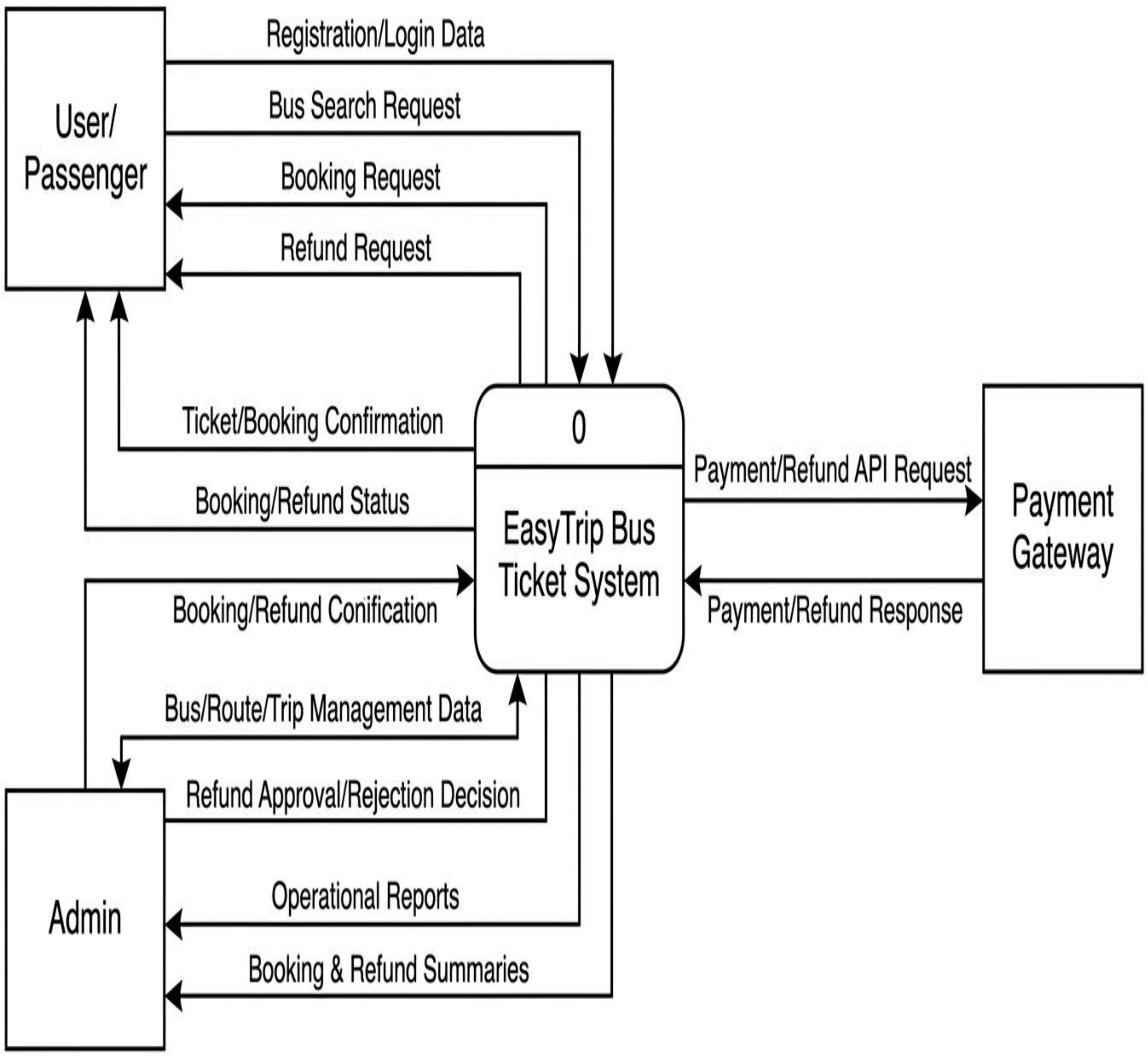


Figure 7.20 DFD Context Level

7.2.2 Level – 1

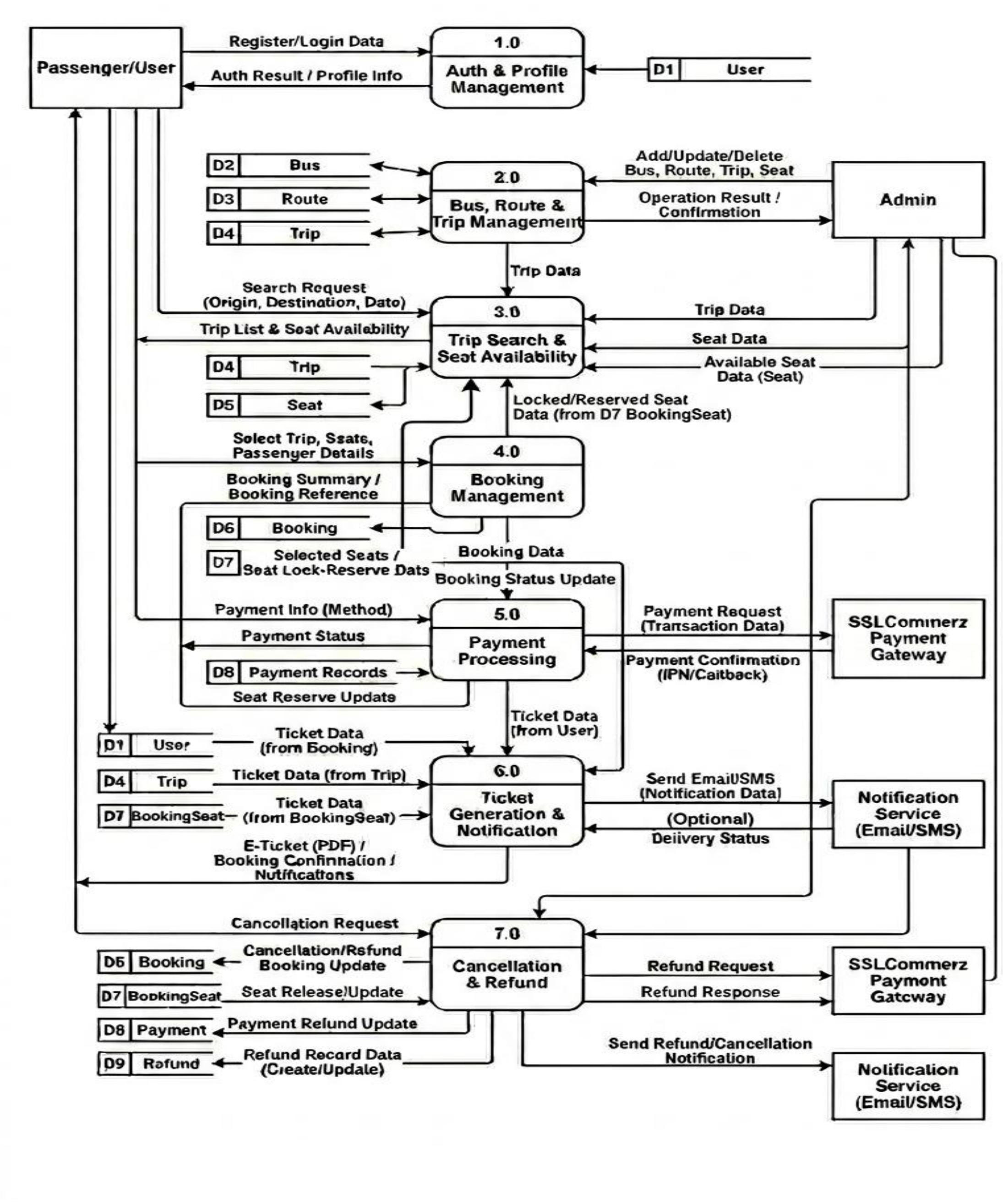


Figure 7.21 DFD Level - 1

7.2.3 Level – 2

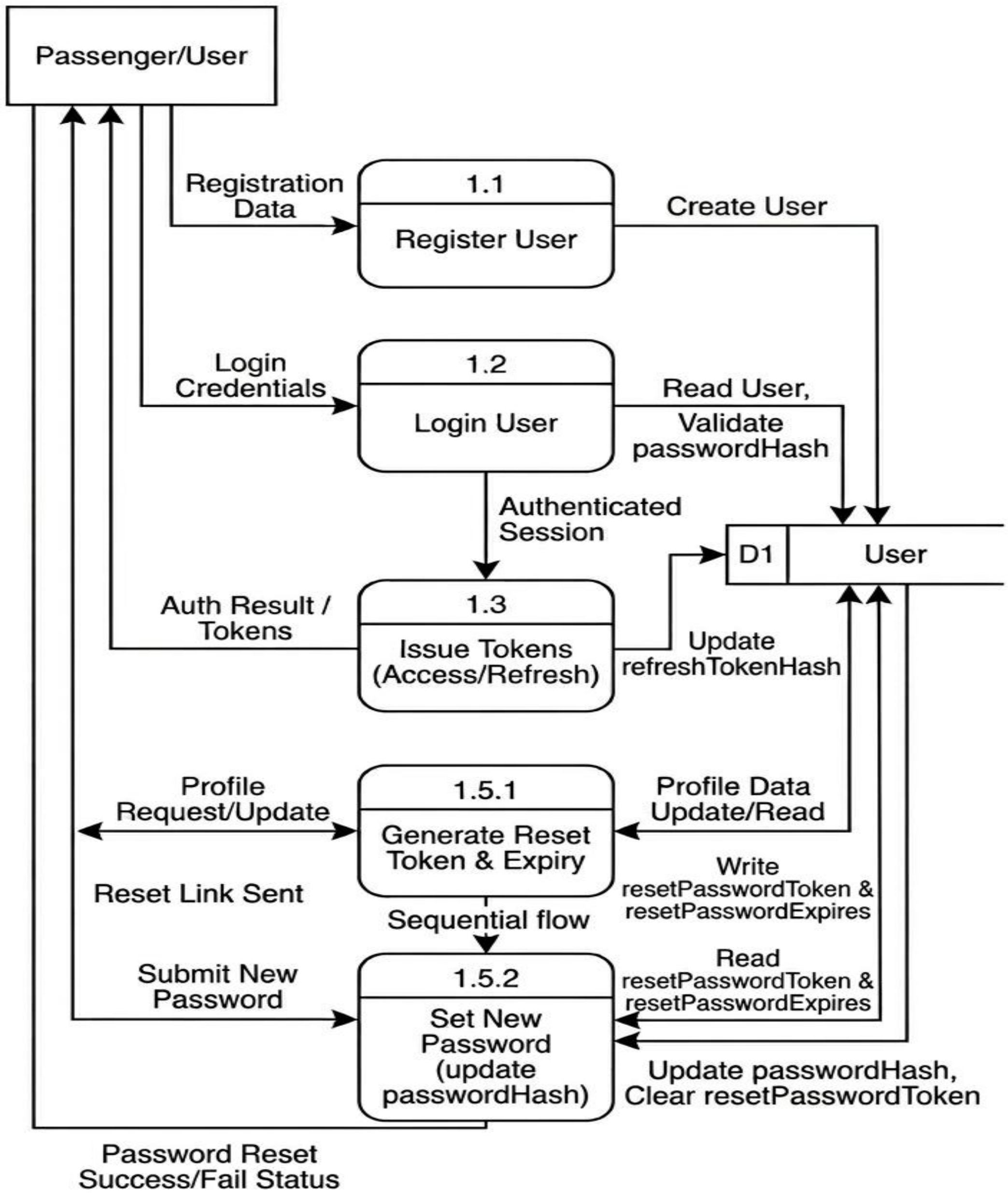


Figure 7.22 DFD Level – 2 Process – 1

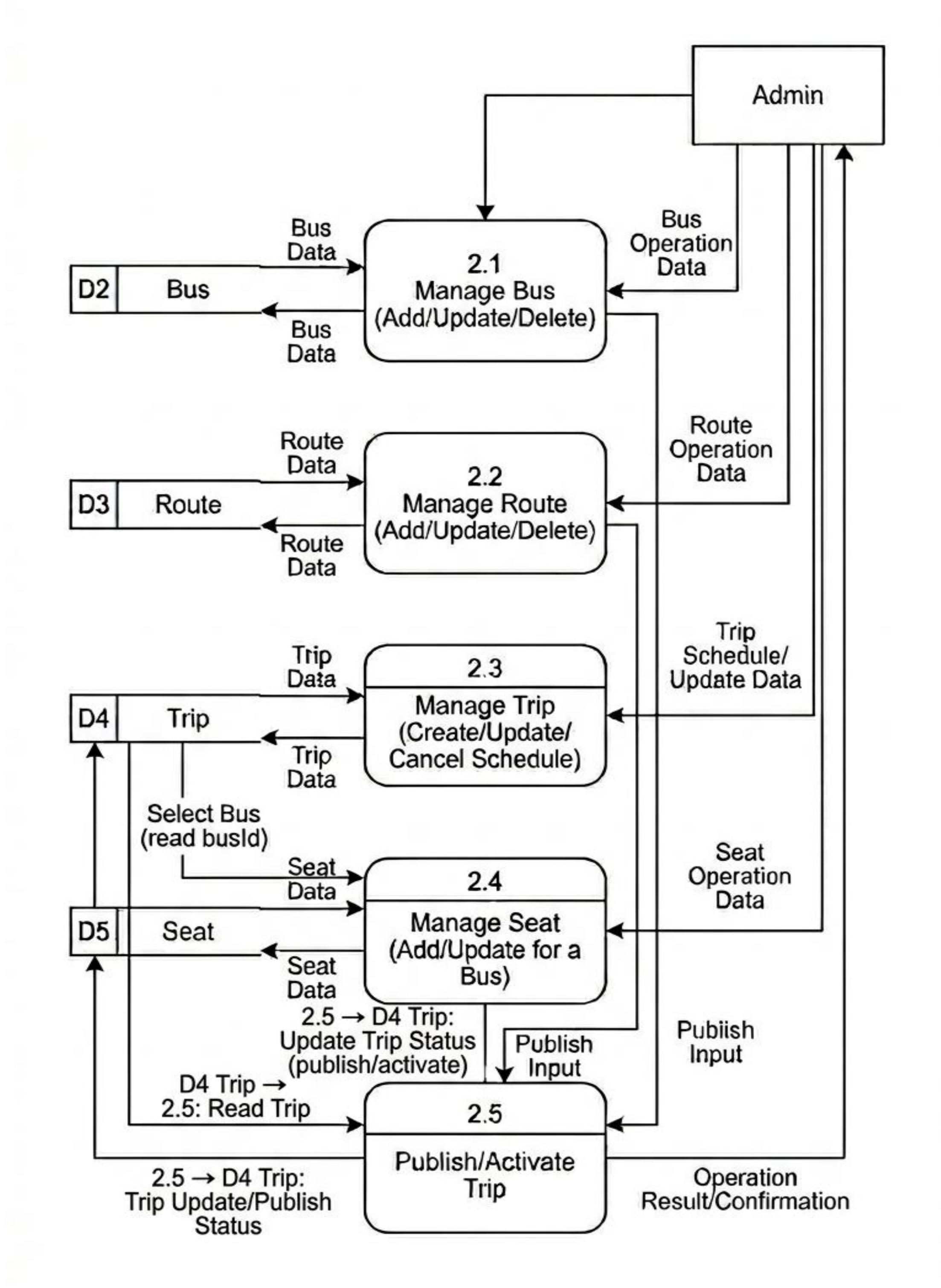


Figure 7.23 DFD Level – 2 Process – 2

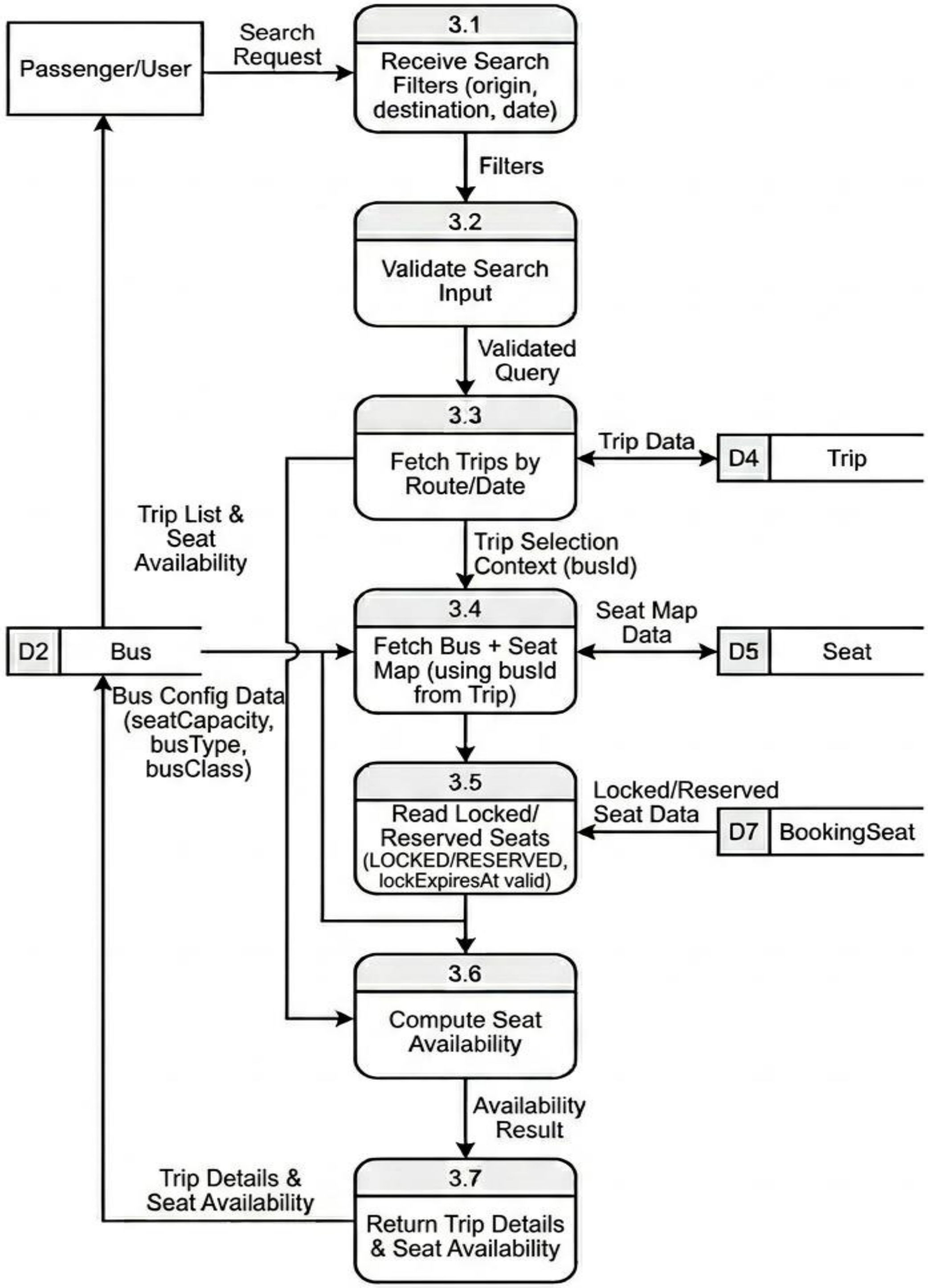


Figure 7.24 DFD Level – 2 Process – 3

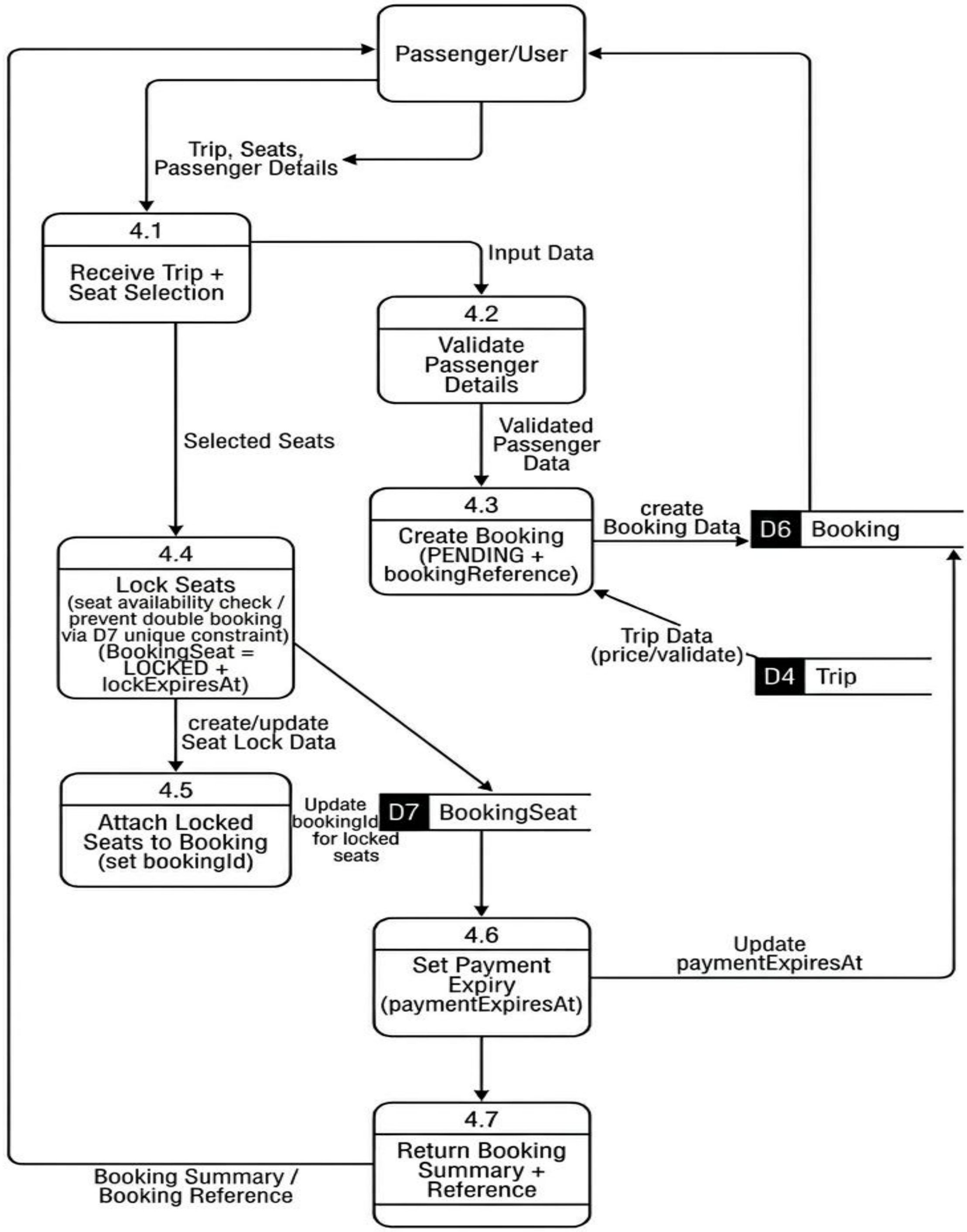


Figure 7.25 DFD Level – 2 Process – 4

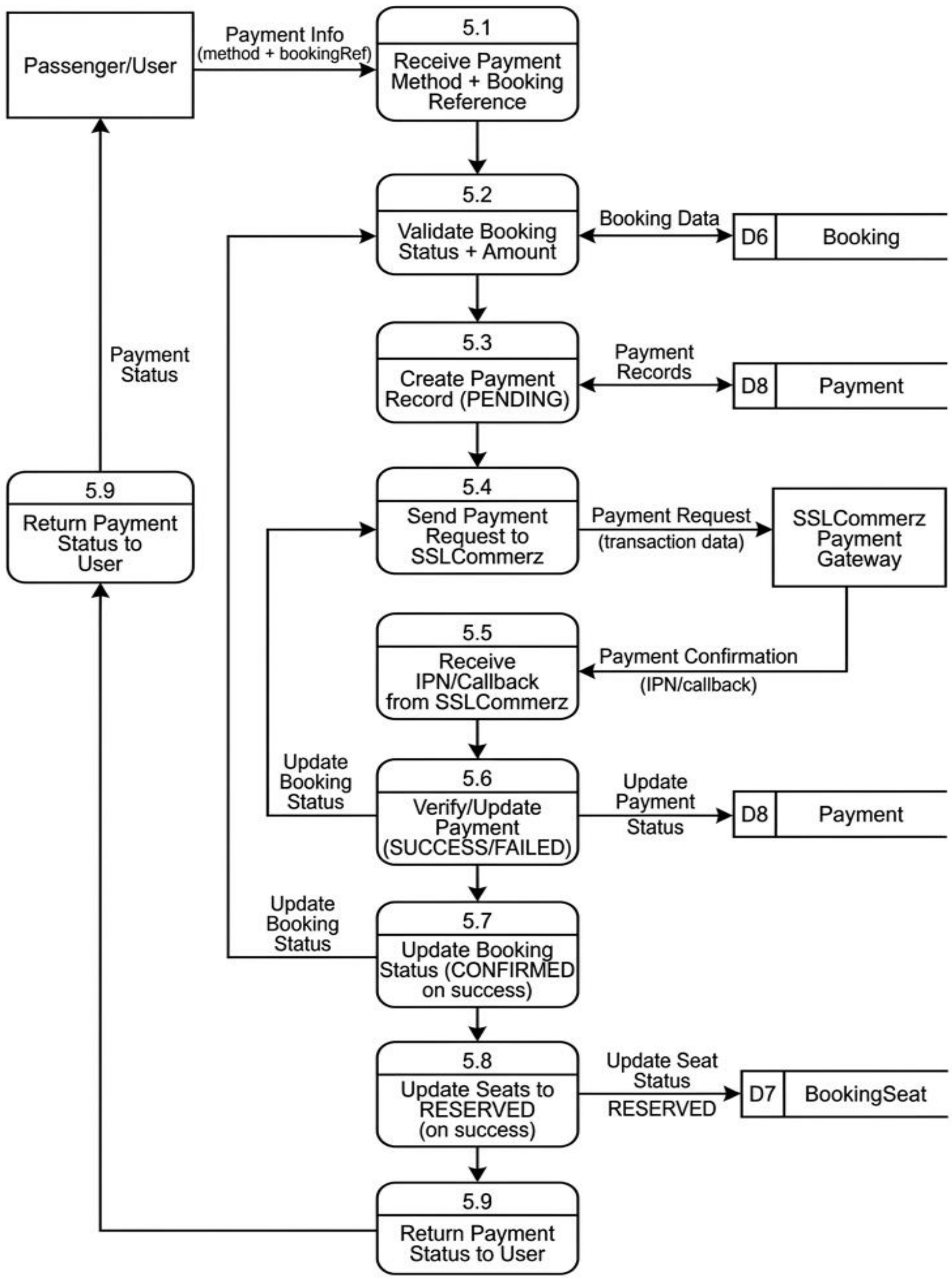


Figure 7.26 DFD Level – 2 Process – 5

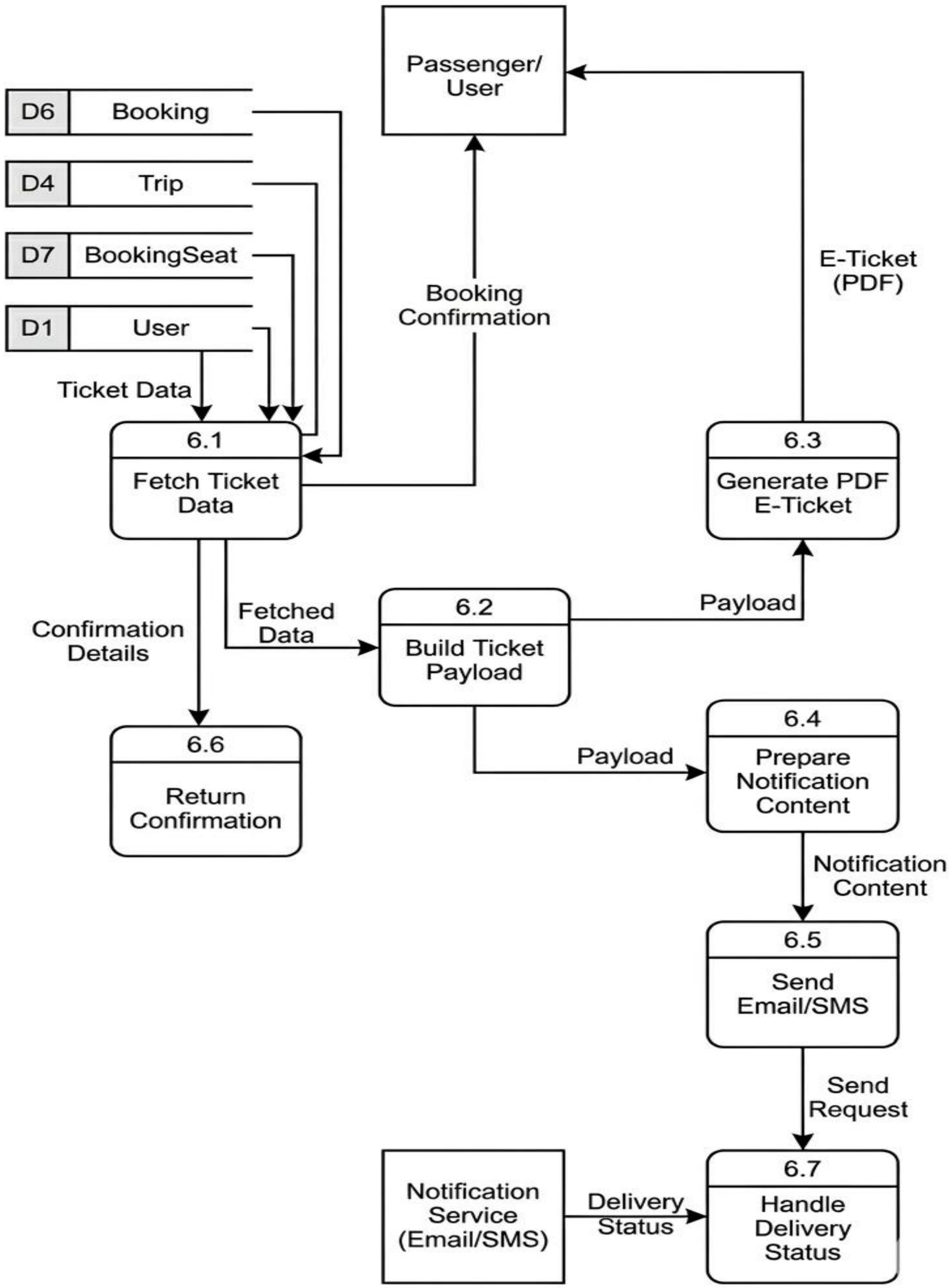


Figure 7.27 DFD Level – 2 Process – 6

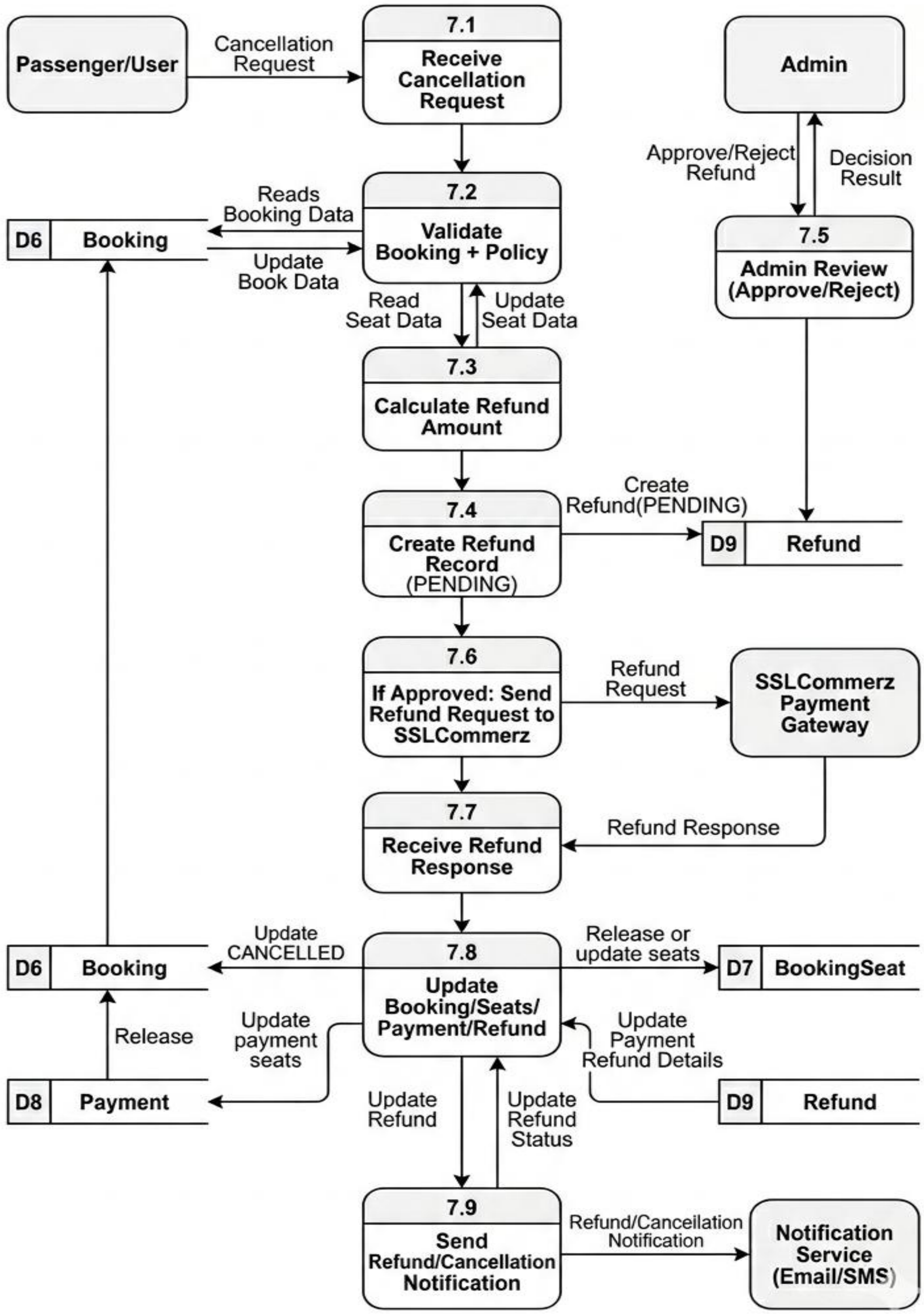


Figure 7.28 DFD Level – 2 Process – 7

7.3 Database Design

7.3.1 Entity Relationship Diagram

An ER Diagram (Entity-Relationship Diagram) is a visual flowchart showing how "entities" (like people, objects, or concepts) connect and relate within a system, primarily used to design and model relational databases by illustrating data structure and relationships using symbols for entities (rectangles), attributes (ovals/text), and relationships (diamonds/lines). The ER Diagaram of our system is given below.

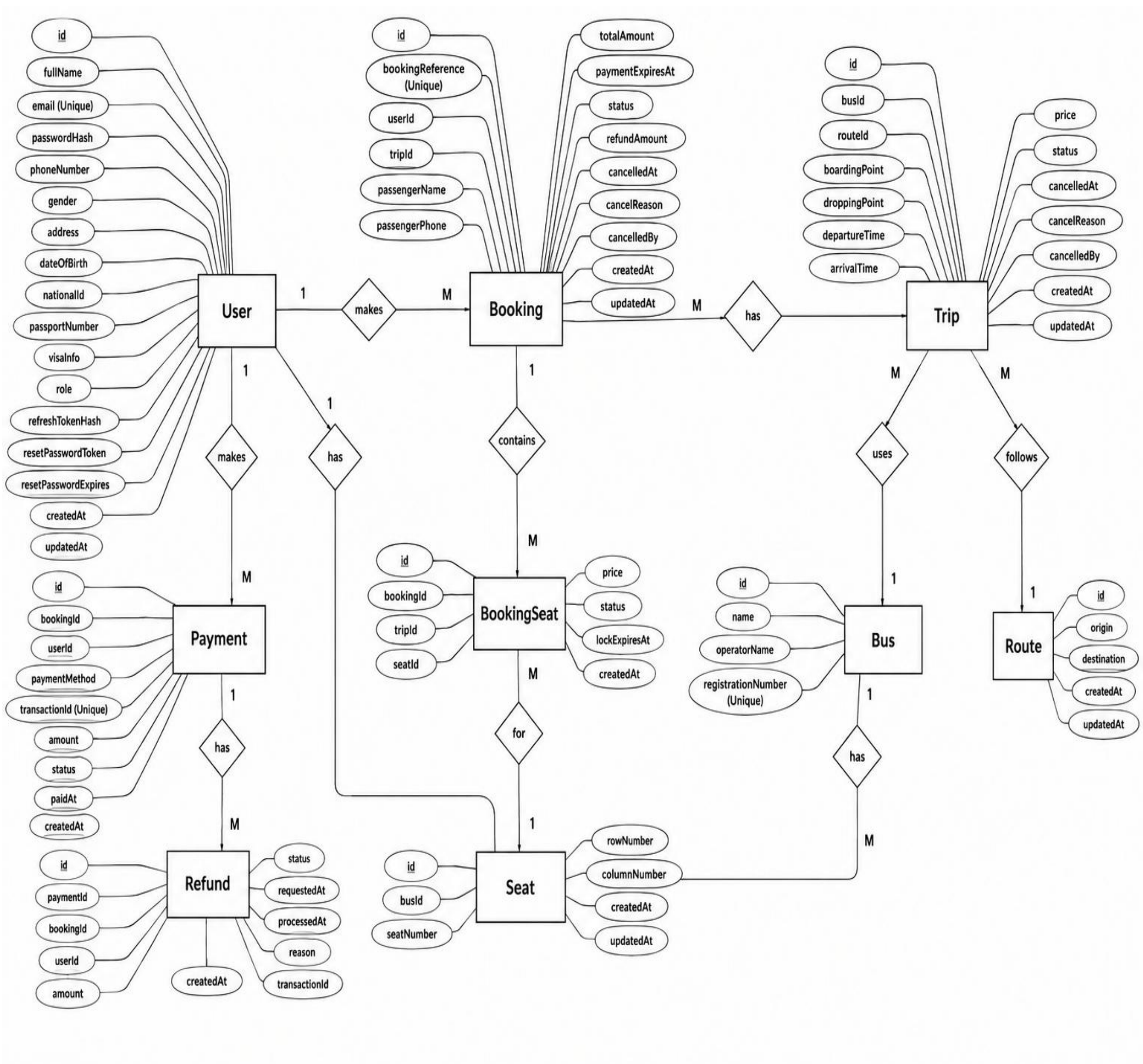


Figure 7.29 ER Diagram of the EasyTrip

Chapter 8.

System Quality & Testing

8.1 System Quality Management

System Quality Management ensures that the **Bus Ticket Management System (EasyTrip)** is reliable, secure, maintainable, and user-friendly before deployment. For a web-based transactional system like EasyTrip, quality is especially important because the platform handles seat availability, bookings, payments, and refunds where errors can directly affect users and business operations.

The quality objectives of EasyTrip are:

- Ensure correctness of core workflows (trip search, seat selection, booking, payment, refund).
- Prevent inconsistent states (e.g., double booking of the same seat).
- Maintain secure access control using JWT authentication and role-based authorization.
- Provide acceptable response time for common user operations.
- Improve maintainability through modular backend and frontend architecture.
- Reduce production defects by integrating testing across modules and flows.

8.1.1 Software Quality Management Process

The quality management process was applied throughout development in iterative cycles.

1. Quality Planning

In this stage, expected quality standards and acceptance criteria were defined.

- ❖ Functional quality goals
  - Booking status transitions must follow business rules.
  - Payment status must correctly reflect gateway callback outcomes.
  - Refund workflow must enforce admin review logic.
- ❖ Non-functional quality goals
  - Secure API endpoints with JWT guards and role checks.

- Data integrity using PostgreSQL constraints and Prisma schema relations.
- Stable API response behavior under normal load.

2. Quality Assurance (Process-Oriented)

Quality assurance focused on preventing defects through process discipline.

- Requirement validation before implementation.
- Modular service/controller design in NestJS.
- DTO-based input validation.
- Peer/self-review of critical logic (booking, payment, refund).
- Consistent error handling and status code policy.
- Environment-based configuration checks.

3. Quality Control (Product-Oriented)

Quality control focused on detecting and fixing defects in implemented features.

- Unit-level verification of service methods.
- API endpoint testing for expected request/response behavior.
- Integration tests for multi-step workflows.
- Manual UI testing for core user journeys in React frontend.
- Regression checks after major fixes.

4. Defect Management and Continuous Improvement

- Defects were logged by module and severity.
- Fixes were validated with targeted re-tests.
- Recurrent defect patterns (e.g., status transitions, edge cases) were addressed by additional validations.
- Final test pass verified end-to-end user and admin workflows.

8.2 Software Test Cases

This section presents representative test cases for major modules. These cases cover authentication, search, booking, payment, refund, and admin control.

Table 23: Test Case Table

TC ID	Module	Test Scenario	Expected Result	Status
TC-01	Authentication	User registration with valid data	Account created successfully	Pass
TC-02	Authentication	Login with correct credentials	User logged in successfully	Pass
TC-03	Authentication	Login with invalid password	Error message displayed	Pass
TC-04	Security	Access protected route without token	Unauthorized response	Pass
TC-05	Trip Search	Search available buses	Matching trips displayed	Pass
TC-06	Seat Selection	View seat availability	Seat map displayed correctly	Pass
TC-07	Booking	Create booking with available seats	Booking created successfully	Pass
TC-08	Booking	Attempt duplicate seat booking	Booking prevented	Pass
TC-09	Payment	Successful payment callback	Payment status updated	Pass
TC-10	Refund	Submit refund request	Refund request created	Pass
TC-11	Admin	Admin approves refund	Refund approved successfully	Pass
TC-12	Validation	Submit invalid booking data	Validation error displayed	Pass
TC-13	Authorization	Normal user accesses admin route	Access denied	Pass
TC-14	Dashboard	Load dashboard statistics	Statistics displayed correctly	Pass
TC-15	End-to-End	Complete booking to payment flow	Workflow completed successfully	Pass

8.2.1 Test Coverage Summary

The test cases were designed to cover:

- **Positive scenarios** (expected valid behavior)
- **Negative scenarios** (invalid credentials, invalid payloads, unauthorized access)
- **Role-based security behavior** (admin vs user boundaries)
- **State transition correctness** (booking/payment/refund statuses)
- **Integration behavior** (payment callback and booking outcome synchronization)

8.2.2 Quality Outcome

From the performed tests, the system achieved the following quality outcome:

- Core modules are functionally stable under normal usage.
- Critical transactional flows (booking-payment-refund) operate consistently.
- Access control and input validation reduce common misuse and unauthorized actions.
- Remaining improvements are primarily optimization-focused (performance tuning, broader automated test suites, and higher concurrency stress tests).

Chapter 9.

Ethical Consideration

9.1 Ethical Considerations in the Software Development Process

Ethics in software development ensures that technology serves people safely, fairly, and transparently. For EasyTrip, ethical practice means protecting users, minimizing harm, and maintaining accountable system behavior.

9.1.1 Data Privacy and Security

EasyTrip stores sensitive information such as user profile data, booking history, and payment transaction references. Ethical handling of this data requires:

- Collecting only necessary data for service delivery (data minimization).
- Protecting authentication and session mechanisms (JWT-based access control).
- Applying secure password handling (hashing, reset token controls).
- Restricting access to privileged operations through role-based authorization.
- Ensuring secure transport and deployment configuration in production.
- Avoiding exposure of sensitive fields in API responses and logs.

From an ethical perspective, users should have confidence that their information is not misused, leaked, or accessed without authorization. Data protection is not only a technical requirement but a trust obligation.

9.1.2 Intellectual Property

The project must respect intellectual property rights at all stages of development:

- Use of open-source libraries must comply with their licenses.
- Third-party integrations (e.g., payment gateway API) must follow official terms and technical policies.
- Design assets, code templates, and documentation references should be properly attributed when required.
- Team-produced source code and documentation should be versioned and ownership clarified.

Ethical IP practice ensures legal compliance and promotes academic integrity and professional credibility.

9.1.3 User Impact

EasyTrip directly affects users' travel planning and financial decisions. Ethical user impact analysis includes:

- Preventing misleading seat availability information.
- Ensuring booking status and cancellation/refund status are clear and understandable.
- Avoiding dark patterns that push users into unintended actions.
- Providing meaningful error messages to reduce confusion and failed transactions.
- Supporting operational reliability to reduce travel disruption.

A user-centered ethical approach requires that system behavior remains predictable, transparent, and recoverable when errors occur.

9.1.4 Bias and Fairness

Although EasyTrip is not an AI-driven recommendation platform, fairness concerns still exist in rule implementation:

- Equal access to booking functionality for all users under the same conditions.
- Consistent cancellation/refund policy enforcement without arbitrary treatment.
- Neutral treatment of routes and schedules in search results based on actual filters.
- No discriminatory behavior based on user profile attributes unrelated to service rules.

Fairness in this context means that business logic applies consistently and transparently to comparable cases.

9.1.5 Responsibility to Stakeholders

The primary stakeholders include passengers, administrators/operators, project supervisors, and hosting/service providers. Ethical responsibility requires balancing stakeholder interests:

- Passengers need reliable booking and privacy.
- Admin/operators need accurate operational tools and traceability.
- Academic/project stakeholders need truthful reporting of system capability and limitations.
- Payment and infrastructure partners require policy-compliant technical integration.

Responsible development avoids overstating readiness, clearly communicates known limitations, and prioritizes high-risk defect mitigation.

9.1.6 Professional Responsibility

Software engineers are expected to act with competence, honesty, and accountability. In EasyTrip, professional responsibility includes:

- Implementing and testing critical workflows before release.
- Maintaining clean, maintainable code and proper documentation.
- Reporting risks and unresolved issues transparently.
- Avoiding shortcuts that compromise security or data integrity.
- Following ethical norms of software engineering and academic conduct.

Professional responsibility ensures the system is not only functional but responsibly engineered.

9.2 Sustainability in the Software Development Process

Sustainability in software development means creating systems that remain useful, maintainable, and beneficial over time with responsible use of resources.

9.2.1 Economic Sustainability

EasyTrip supports economic sustainability through both development and operational choices:

- Use of open-source technologies (React, NestJS, PostgreSQL, Prisma) reduces licensing cost.
- Modular architecture lowers long-term maintenance cost by simplifying updates.
- Digital workflows reduce manual processing overhead in ticketing operations.
- Better booking accuracy and reduced operational errors minimize avoidable financial loss.
- Scalable deployment options allow growth without immediate high capital expenditure.

From a project perspective, this makes EasyTrip feasible for gradual adoption by small-to-medium transport operations and suitable for iterative enhancement.

9.2.2 Social Sustainability

Social sustainability focuses on long-term user and community value:

- Improves service accessibility by enabling online trip discovery and booking.
- Reduces stress and uncertainty for passengers through clearer information and status tracking.
- Encourages transparent handling of cancellations and refunds.
- Supports administrative efficiency, which can improve service consistency for the public.

In addition, a user-friendly and secure ticketing platform can contribute to broader digital inclusion by helping people access reliable transport services with less friction.

Chapter 10.

Conclusion

This chapter presents the overall conclusion of the **Bus Ticket Management System (EasyTrip)** project by summarizing the developed system, its practical benefits, limitations, learning value of the practicum, and future enhancement directions.

10.1 Brief Overview of the Project

The **Bus Ticket Management System (EasyTrip)** was developed as a web-based software solution to modernize and simplify bus ticketing operations. The project addressed common issues in traditional or semi-manual ticketing methods, including inefficient trip discovery, seat allocation conflicts, booking management complexity, and weak transaction visibility.

The system was implemented using:

- **Frontend:** React.js
- **Backend:** NestJS
- **Database:** PostgreSQL with Prisma ORM
- **Security:** JWT-based authentication and role-based access control

Core functional modules include:

- User authentication and profile handling
- Route, bus, and trip management
- Trip search and seat selection
- Online booking workflow
- SSLCommerz payment gateway integration
- Refund request and processing flow
- Admin dashboard and operational controls

Overall, the project produced an end-to-end transactional platform that supports both passenger-facing and administrator-facing operations.

10.2 Proposed System Benefits

The proposed EasyTrip system provides several practical benefits for users and operators:

- **Improved accessibility:** Users can search trips and book tickets online without physical counter dependency.
- **Better operational efficiency:** Admins can manage routes, trips, and bookings from a centralized system.
- **Reduced manual errors:** Structured workflow and database constraints help reduce seat conflict and record inconsistency.
- **Transparent transaction flow:** Booking, payment, and refund statuses are tracked clearly, improving trust.
- **Enhanced security and control:** JWT authentication and role-based authorization protect restricted operations.
- **Scalable architecture:** Modular full-stack design supports incremental feature upgrades over time.
- **Data-driven management potential:** Dashboard metrics provide a foundation for operational decisions.

These benefits demonstrate that the system can improve service quality and management reliability in practical transport contexts.

10.3 Limitations of the Project

Although the system is functionally complete for core operations, some limitations remain:

- **Limited deployment scale validation:** High-concurrency stress testing was not fully performed in production-like environments.
- **Payment dependency scope:** Gateway behavior and availability depend on third-party service constraints.
- **Advanced analytics not fully implemented:** Current dashboard focuses on operational aggregates rather than predictive insights.

- **Notification ecosystem is basic:** Full multi-channel notification support (SMS/push) can be expanded.
- **Operational policy variation:** Real transport businesses may require additional policy customization for refunds/cancellations.
- **Infrastructure hardening scope:** Enterprise-level resilience practices (advanced monitoring, failover automation) are future work.

These limitations are typical for an undergraduate practicum project and provide clear direction for further development.

10.4 Practicum and Its Value

The practicum provided significant academic and professional value by connecting theoretical software engineering concepts to a real-world domain problem. Key value outcomes include:

- **Application of software engineering lifecycle:** Requirements, design, implementation, testing, and documentation were completed in a structured way.
- **Hands-on full-stack development experience:** Integration of frontend, backend, database, and security mechanisms.
- **Understanding of transactional workflow design:** Practical handling of booking/payment/refund state transitions.
- **Exposure to third-party integration:** Realistic payment gateway callback and status handling logic.
- **Improved problem-solving and debugging ability:** Addressing edge cases in seat allocation and status consistency.
- **Team/project discipline:** Better planning, module decomposition, and iteration-based delivery.

Thus, the practicum was not only a project outcome but also a strong learning process that strengthened technical competence and engineering judgment.

10.5 Future Plan

Future development can extend EasyTrip into a more production-ready and feature-rich platform. Planned enhancement areas include:

- **Scalability and performance improvements** through caching, query optimization, and load testing.
- **Advanced analytics dashboard** with route demand trends, revenue intelligence, and operational forecasting.
- **Stronger notification system** (email + SMS + push) for booking, cancellation, and refund events.
- **Enhanced customer features** such as e-ticket QR validation, loyalty programs, and saved traveler profiles.
- **Improved policy engine** for configurable cancellation/refund rules per operator or route type.
- **Multi-operator support** for broader marketplace-style deployment.
- **Security and observability upgrades** including deeper audit logs, monitoring, and alerting integration.
- **Mobile-first expansion** via dedicated mobile applications or progressive web app enhancements.

These future plans indicate that EasyTrip has a sustainable foundation and can evolve into a robust digital transport platform beyond the current practicum scope.

References

React (2025) React Documentation. Available at: React Official Documentation (Accessed: 18 March 2026).

NestJS (2025) NestJS Documentation. Available at: NestJS Official Documentation (Accessed: 27 February 2026).

PostgreSQL (2025) PostgreSQL Documentation. Available at: PostgreSQL Official Documentation (Accessed: 9 April 2026).

Prisma (2025) Prisma ORM Documentation. Available at: Prisma Official Documentation (Accessed: 14 March 2026).

SSLCommerz (2025) SSLCommerz Developer Documentation. Available at: SSLCommerz Official Website (Accessed: 2 May 2026).

Node.js (2025) Node.js Documentation. Available at: Node.js Official Documentation (Accessed: 21 April 2026).

TypeScript (2025) TypeScript Documentation. Available at: TypeScript Official Documentation (Accessed: 11 February 2026).

Tailwind CSS (2025) Tailwind CSS Documentation. Available at: Tailwind CSS Official Documentation (Accessed: 30 March 2026).

GitHub (2025) GitHub Documentation. Available at: GitHub Official Website (Accessed: 6 May 2026).

Postman (2025) Postman Learning Center. Available at: Postman Official Website (Accessed: 25 February 2026).